

XYZ MGMT Device Farm Engineering & Operations Handbook

A sectioned technical reference for physical device fleets, iOS/Android control, networking, client separation, content operations, reliability and implementation

Internal research edition

27 August 2026

Table of Contents

Reader Notice

This handbook is a **technical and operational engineering reference** for an organization-owned physical-device fleet used for authorized creator/social-media operations, testing, content production and related workflows. It incorporates internal research, hands-on observations, vendor documentation, platform/OS documentation, and earlier engineering work.

It deliberately separates **mechanism** from **mythology**. A claim made by an operator is not converted into a fact merely because it is repeated. A vendor feature is not treated as proven until it is tested on the actual hardware. A platform behavior observed once is not treated as a universal algorithm rule. Every architecture-blocking uncertainty becomes an experiment.

This handbook does **not** provide an operational playbook for bypassing platform enforcement, manufacturing fake engagement, mass-creating replacement identities, falsifying geolocation, fingerprint spoofing, scraping data in violation of service terms, or automating prohibited third-party UI behavior. Networking, proxies, device identity and automation are explained for legitimate segmentation, testing, privacy, observability and authorized fleet operation.

Evidence legend

Label

Meaning

Engineering use

VERIFIED-FIRST-PARTY

Supported by current OS/platform documentation

May be used as architecture input, subject to version changes

INTERNAL-MEASURED

Observed by XYZ/Dim on owned hardware

Strong for the exact setup tested; do not overgeneralize

VENDOR-CLAIM

Vendor states the capability

Must be bench tested before procurement/production dependence

OPERATOR-CLAIM

External operator statement

Hypothesis/economic input only unless corroborated

INDUSTRY-AGGREGATE

Repeated community/creator claims

Useful for experiment design, not production constants

INFERENCE

Derived analysis from multiple facts

Useful when assumptions are explicit and falsifiable

OPEN

Not resolved

Architecture cannot depend on it until tested or verified

POLICY

Chosen organizational boundary

Enforced even if a different technical path exists

Source precedence

When sources disagree, use this order:

internal bench measurement for the exact hardware/software version being built;

current first-party OS/platform documentation;

maintained open-source implementation documentation;

vendor technical documentation;

independent engineering reports;

operator testimony;

forums/marketing folklore.

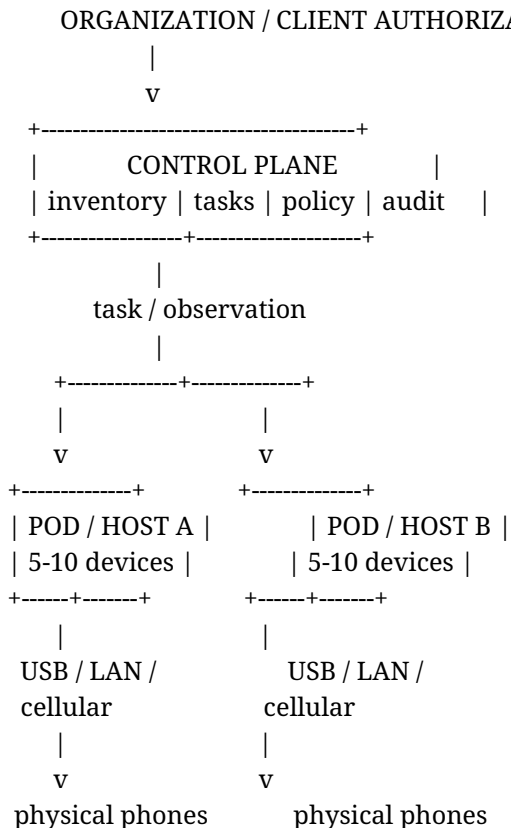
This does **not** mean a bench observation proves a universal platform rule. It means it is the most relevant evidence for whether *our exact hardware path works*.

Executive Synthesis

What this project actually is

[INFERENCE] The phrase "phone farm" hides the real engineering problem. The system is a compact private cloud whose compute nodes happen to be phones. It has power distribution, network interfaces, stateful workloads, identity, storage, scheduling, telemetry, operators and failure domains. Treating it like "20 phones on a hub" is equivalent to treating a server rack as "20 boxes plugged into a switch."

A useful system model is:



The scaling unit is the **pod**, not the individual phone and not a 100-device monolith. A pod has a bounded power domain, host, hub topology, network policy and failure radius. This makes capacity measurable and incidents recoverable.

The most important updated conclusions

1. The Mac mini is worth testing first, but the key USB-display claim is still OPEN

Dim's hands-on notes correctly identify a Mac mini as attractive because it is locally available, can run Apple Configurator, supports WDA/XCUITest if that route is accepted, and avoids relying on an unsigned third-party binary for some Apple workflows.

However, [OPEN] current Apple public documentation does not establish that a tethered iPhone screen is exposed as a scalable public AVFoundation capture source for many simultaneous devices. QuickTime's ability to display a connected iPhone is not enough evidence for a 20-device programmatic architecture. The handbook therefore makes **USB screen capture at scale a Gate-1 experiment**, not a promise.

2. Display and input must be designed independently

Dim's Xiaowei test is valuable because it proves the conceptual separation: the PC could see the phone only through same-network mirroring; it could not automatically control it. The main input options remain external HID/control hardware or a signed on-device automation component such as WebDriverAgent.

[VERIFIED-FIRST-PARTY] Apple's UIKit documentation matters here: from iOS 17 onward, an app that has not opted out of indirect input support receives pointing-device clicks as indirectPointer, whereas pre-iOS-17 compatibility defaults behaved as direct touch. This proves that pointer input **can be distinguishable at the OS API**. It does not prove any social platform uses the signal.

3. Reset is not a universal identity eraser on either iOS or Android

[VERIFIED-FIRST-PARTY] Apple DeviceCheck persists two developer-defined bits and a timestamp per device across app reinstall, device transfer and Erase All Contents and Settings. [VERIFIED-FIRST-PARTY] Google's 2026 Play Integrity device-recall beta similarly lets participating apps recall custom per-device state after reinstall and device reset.

Therefore the correct architecture statement is not "iOS is permanently fingerprinted" or "Android resets cleanly." It is: **ordinary raw identifiers and backend anti-abuse state are different mechanisms**. A system must not base asset economics on the assumption that a reset clears all server-side device state.

4. Client isolation is a first-class architecture problem

The new account-separation research changes the priority order. Shared business containers, credential custody, session bleed, billing entanglement, poor staff offboarding and vendor concentration create a larger practical blast radius than exotic fingerprint theories.

[INFERENCE] The system should model Client, Authorization, CredentialReference, PlatformAsset, and OperatorRole as core database entities rather than metadata attached later.

5. "Warmup" is not one variable

The Instagram warmup source contains directly contradictory successful practices: immediate posting, several-day delays, 14-day routines, and month-long routines. Proxy and device claims conflict as well. This is evidence that the folk concept "warmup" bundles many different variables.

[INFERENCE] A better model is:

observed outcome = f(account provenance,
policy/recommendation eligibility,
content quality,
audience fit,
account age,

activity history,
current platform enforcement,
operator behavior,
random/recommendation variance)

The handbook therefore treats warmup advice as **experiment hypotheses**, not a clock-based SOP.

6. The first milestone is not growth; it is deterministic operation

Before scaling traffic, the system must prove: reliable enumeration, device identity, screen/health observation, controlled input on owned test devices, authorized media delivery, task execution, remote recovery, power/thermal stability, audit and operator separation.

A strong pilot exit criterion is not "it posted once." It is **twenty consecutive unattended task cycles with correct attribution and no unexplained device loss**, followed by a soak test and recovery test.

Section Map

Section	
Topic	
Main question answered	
0	
Orientation, evidence, boundaries	
What do we know, and how confident are we?	
1	
System model & operating choices	
What are we actually building?	
2	

Physical plant

How do the devices stay powered, cool, connected and serviceable?

3

iOS engineering

How do iPhones identify, provision, display, accept input and ingest media?

4

Android engineering

What is easier/different on Android, and what integrity assumptions are unsafe?

5

Networking

How do local control and Internet egress work without conceptual mistakes?

6

Client/account separation

How do we keep one client/staffer/vendor failure from becoming everyone else's?

7

Content & platform evidence

What is documented, what is measured, and what is folklore?

8

Control plane & remote ops

What software coordinates the fleet and humans?

9

Reliability, security, incidents

How does the farm fail safely and observably?

10

Experiments, pilot & scaling

What must be proven before purchasing/scaling?

11

Claude/engineering execution

What exactly should the implementation agent build first?

SECTION 1 - System Model and Architecture Choices

A phone farm is only useful when the business process around it is explicit. This part defines the OFM use case, separates legitimate fleet operations from manipulation, and establishes the architecture used throughout the manual.

1. Phone farms in OFM

In an OFM context, a phone farm is a managed collection of physical mobile devices used by an agency or creator team to operate authorized social accounts, prepare and publish content, review notifications, collect performance signals and support traffic acquisition. The physical phones are execution endpoints; the real system is the combination of devices, people, network, content rights, credentials, workflow software and analytics.

Industry usage and terminology

Public operator communities use “phone farm” loosely. It can mean ten old iPhones on a desk, a rack of Android phones, a motherboard rack without screens, a set of remotely controlled devices, or a larger operation with coordinators and staff. These reports are anecdotal and should not be treated as evidence that a particular platform will tolerate the associated behavior.

The legitimate engineering interpretation is closer to a corporate mobile fleet or device lab: each device is inventoried, assigned, secured, connected to controlled networks, monitored and maintained. The difference is that the workload is creator-content operations rather than QA testing or retail kiosk use.

- Define the owner of every device and social account.
- Treat creator authorization and content rights as first-class records.
- Separate device administration from social publishing.
- Measure output with business metrics rather than “number of phones.”

What the farm is not

A physical phone is not an invisibility device. Platforms can evaluate account behavior, content similarity, network characteristics, login history and many other signals. A real handset may make the workflow more operationally convenient, but it does not convert prohibited behavior into permitted behavior.

The architecture should therefore optimize for repeatability, security, operator throughput, reliability and attribution. It should not optimize for hiding the existence of the fleet.

- Do not buy hardware before mapping the traffic funnel.
- Do not assume one physical device must equal one account. The correct mapping depends on authorization, platform rules, staffing and operational risk.

Engineering checklist

- Write down the business goal for each platform.
- List authorized creator accounts and account owners.
- Define which actions are manual, API-driven or scheduler-driven.
- Create a platform-rule register with review dates.

2. Traffic funnel and business model

The phone-farm layer sits in the middle of a larger creator acquisition funnel. Engineering decisions should be evaluated by their effect on measurable funnel stages rather than raw posting volume.

Reference funnel

A typical funnel is: approved creative asset -> platform post -> profile visit -> tracked link or landing page -> creator page -> subscription or other paid action. Depending on the business, there may be additional layers such as a link hub, geofenced landing page, email capture, or CRM attribution. Every added layer creates potential conversion loss and additional privacy/security obligations.

The control plane should attach campaign, creator, platform, account, device, operator and asset identifiers to each publishing task. The analytics plane should connect those identifiers to post-level reach where available and to downstream click/conversion events where lawful and technically possible.

- Primary operational metrics: task completion rate, post success rate, account availability, content approval latency.
- Primary marketing metrics: reach, profile visits, click-through rate, landing conversion, subscriber conversion, revenue per thousand impressions where calculable.
- Primary reliability metrics: device heartbeat, battery/thermal status, network packet loss, API error rate, queue age.

Unit economics

A useful model treats each account or campaign as a cost center. Hardware amortization, software, connectivity, employee time, content production and creator revenue share should be visible. “More phones” only creates value if incremental gross contribution exceeds incremental infrastructure and labor cost.

Attribution is imperfect because not every platform exposes the same analytics and some downstream conversions cannot be deterministically tied to a post. The system should distinguish observed events from modeled or inferred attribution.

Layer

Example measure

Engineering owner

Creative

Approved assets per week; QC rejection rate

Content ops

Publishing

Task success; queue age; retries

Fleet/backend

Platform

Reach; profile actions
 Growth ops
 Landing
 Unique clicks; conversion
 Web/backend
 Revenue
 Paid conversion; ARPU/LTV
 Finance/analytics

3. What physical phones solve - and what they do not

Physical devices are valuable when the workflow actually requires the native mobile app, hardware-bound capabilities, mobile camera/media access, or a standardized operator interface. They are less valuable when an official API or web console already provides the required function.

Figure 3-1. Control-plane and data-plane separation.

Problems physical devices solve

Phones provide the authentic application runtime, camera roll, push notifications, biometric-secured credential storage, mobile-only creation tools and native moderation/status screens. Organization-owned devices can also be physically segmented from employees' personal phones.

For Android, corporate devices can be fully managed or dedicated under Android Enterprise. For Apple fleets, organization-owned supervised devices can receive stronger MDM controls than unsupervised personal devices. [S1][S5]

- Native-app-only features.
- Consistent operator work surface.
- Separation between personal and corporate credentials.
- Physical inventory that can be wiped, reassigned and retired.

Problems they do not solve

Phones do not remove platform terms, content restrictions, API rules or behavioral enforcement. They also do not automatically solve account ownership, credential leakage, employee mistakes, attribution or creator-consent management.

A rack can even make risk worse if the same content is repetitively distributed, credentials are broadly shared, devices are not patched, or network architecture creates one large blast radius.

- Build authorization and access control before scaling hardware.
- Use official publishing APIs where available and suitable.
- Keep humans in approval loops for ambiguous or high-risk actions.

4. Platform-policy and authorization boundary

The most important non-hardware requirement is an explicit rule boundary. Different platforms permit multiple accounts, third-party management, APIs and scheduling to different degrees, but commonly prohibit spam, fake engagement, coordinated inauthentic activity, automated bulk account operation, or circumvention of enforcement. These rules can change.

Current examples from first-party rules

TikTok's integrity rules distinguish authentic multi-account use from deceptive or spam behavior and prohibit automation used to operate accounts in bulk or manipulate platform systems. TikTok also provides a formal Content Posting API for authorized applications, with scopes and audit requirements. [S22][S23]

Reddit's rules require authentic participation and prohibit spam and manipulation. Snapchat explicitly prohibits spam and artificially inflated engagement. X permits legitimate third-party account management but prohibits coordinated inauthentic activity, duplicate amplification and ban-evasion behavior. [S24][S26][S27]

Authorization records

For every creator and account, retain a signed or otherwise auditable authorization record stating who owns the account, what the agency may do, which employees or systems may access it, what content is approved, and what happens on termination. The agency should not rely on informal chat messages as its only control evidence.

Keep platform credentials separate from legal identity documents and sensitive creator files. The system should store credential references to a vault rather than passwords in application tables or spreadsheets.

- Creator ID and contract/authorization reference.
- Account owner and recovery email/phone owner.
- Permitted actions and publishing platforms.
- Content usage scope and expiration/revocation process.
- Offboarding/wipe/recovery procedure.

5. Architecture decision: API-first, device-first or hybrid

The correct architecture depends on which functions are officially exposed by platform APIs and which require a person in a native application. A hybrid design is usually the most maintainable: APIs or approved schedulers handle supported publishing and analytics; physical phones handle native-only tasks and human review.

Figure 5-1. Hybrid reference architecture.

API-first

Use an API-first workflow when the platform provides a supported publishing or analytics endpoint and the agency can meet app-registration, scope and audit requirements. API-first systems are easier to log, retry, rate-limit, test and attribute. TikTok's Content Posting API is one example of an explicit, authorized posting path. [S22]

- Advantages: deterministic requests, auditability, fewer physical touch points, easier scaling.
- Costs: developer integration, platform app review, scope limitations, changing APIs.

Device-first

Device-first means a human operator performs most social actions on managed phones. This is slower to scale but can be appropriate for native-only workflows. Fleet software should coordinate assignments and record outcomes without pretending to be the human.

If the platform does not permit automated UI operation, do not turn accessibility tools, ADB input commands or computer vision into a covert substitute for an official API.

- Advantages: access to native UX; human judgment.
- Costs: labor, training, ergonomic complexity, more devices, weaker deterministic automation.

Hybrid

Hybrid systems place a coordinator above both paths. A task can route to an official API executor, an approved scheduler, or a human/device queue depending on platform, account and action type. All paths write back a common result schema.

Dimension

API-first

Device-first

Hybrid

Scale

High where API permits

Labor-bound

High for supported actions

Auditability

Strong

Requires operator capture

Strong if unified

Native-only features

Limited

Strong

Strong

Platform dependence

API contracts

App UX changes

Both

Recommended role

Default where supported

Exception/native tasks

Overall architecture

SECTION 2 - Physical Plant: Devices, Racks, Power, USB and Thermal

A rack of phones is a continuous electrical and thermal load, not a collection of casual chargers. This part treats the installation as small infrastructure: load planning, serviceability, airflow, cable discipline and lifecycle management.

6. Device selection

Device choice should be driven by supported OS life, battery condition, application compatibility, management support, repairability and total cost rather than the cheapest used handset price.

Selection criteria

Prefer models that remain inside the OS/security support window for the expected service life. Confirm the required social apps still support the OS version. For Android, verify Android Enterprise management capabilities and consider zero-touch eligible procurement for larger deployments. For Apple, organization-owned devices that can be enrolled and supervised through the intended deployment path are easier to administer consistently. [S1][S5]

Used devices can lower capital cost but introduce unknown battery health, replaced parts, activation locks, inconsistent storage and shorter remaining update life. A receiving inspection process is therefore mandatory.

- Supported OS/security updates for planned life.
- 64 GB or more storage if local media caching is heavy; size to actual workflow.
- Healthy battery and charging port.
- Stable Wi-Fi radios and compatible bands.
- Consistent model families to reduce spare-parts and cable variety.
- No activation lock, unknown MDM enrollment or carrier financing lock.

Screened vs headless hardware

Phone-motherboard racks and screenless devices appear in operator communities, but they add maintenance and compliance uncertainty because the hardware may be modified or difficult to recover. For a professional pilot, normal complete phones are easier to inventory, reset, update, visually inspect and hand to an operator.

Only consider custom hardware after the software architecture and workload are proven on standard devices.

Factor

Pilot preference

Scale preference

Model mix

1-2 models max

Standardize by cohort

OS age
 Current or recent
 Long remaining support
 Battery
 Known healthy
 Replaceable service process
 Procurement
 Reputable used/new
 Enterprise/reseller where possible
 Management
 Manual + MDM/Enterprise
 Automated enrollment

7. Rack and mechanical design

The mechanical objective is not maximum density. It is stable airflow, easy identification, fast replacement, safe cabling and enough access that an operator can service one phone without disturbing ten others.

Figure 7-1. Example serviceable rack layout.

Slot geometry

Use vertical or slightly tilted slots so screens remain visible and warm air can rise between devices. Leave air gaps instead of clamping phones tightly together. The required spacing depends on device power, ambient temperature and fan airflow; validate it with temperature measurements rather than a fixed internet rule.

Every slot should have a stable slot ID. The physical label, inventory database and cable label should use the same identifier. Avoid relying on “third phone from the left.”

- Front access to power button and charging connector.
- No sharp edges or pressure on battery area.
- Cable bend radius and strain relief.
- Fan guards and removable dust filters where appropriate.
- A service aisle or pull-out tray for dense racks.

Material choices

Nonconductive trays simplify electrical isolation. Metal enclosures can provide stronger structure but require proper grounding and careful cable protection. 3D-printed fixtures should be made from suitable materials and kept away from hot power components; do not assume hobby plastics are fire-rated.

Keep chargers, AC distribution and high-current DC distribution in a power zone separate from phone slots. This improves cooling and makes fault isolation easier.

Engineering checklist

- Assign one immutable slot ID to every physical position.
- Provide a safe way to remove one device without pulling adjacent cables.
- Measure inlet and exhaust temperature under full load.
- Document rack weight, power-feed rating and emergency disconnect location.

8. Power architecture

Power should be designed from measured load. Charging power is dynamic: a phone may draw more while recovering from low battery, then taper as it approaches full charge, and thermal controls can reduce charging rate. USB Power Delivery negotiates power between compatible source and sink; USB PD 3.1 supports substantially higher power levels than phones normally require, so charger nameplate wattage is not the same as actual phone draw. [S10]

Figure 8-1. Illustrative power scaling model; use measured values for design.

Load model

For planning, define P_{devices} as the sum of measured average phone power, P_{peak} as the likely coincident peak, and P_{infra} as routers, switches, APs, controller PCs, fans and monitoring. Select supplies, branch circuits and UPS capacity with engineering headroom rather than operating continuously at nameplate limits.

Illustrative example only: if 25 phones average 5 W while connected, device load is about 125 W. Add the actual measured infrastructure load and then add headroom. This example is not a specification for any handset.

- Measure at the AC input with a power meter during representative use.
- Record both idle/maintenance load and recovery-from-low-battery load.
- Do not assume all multi-port chargers can deliver their advertised maximum to every port simultaneously.
- Use chargers/cables from reputable vendors with appropriate certifications for your region.

Power zoning

Split large racks into independently protected zones. A single failed adapter or tripped protection device should not remove the whole fleet. Network, control server and essential monitoring may deserve UPS runtime even if phone charging itself is not fully UPS-backed.

If chargers expose smart power allocation, test how port behavior changes when many devices are connected or disconnected at once.

Engineering checklist

- Create a port-to-slot map.
- Label every power supply with zone and rated output.
- Avoid improvised high-current wiring.
- Document shutdown/startup sequence after utility failure.

9. USB topology and cabling

USB can carry power, data or both. A rack that uses ADB over USB or Apple configuration tooling has different requirements from a charge-only rack. Treat the USB data topology as an engineered bus, not an unlimited fan-out.

Data vs charge

Charge-only cables reduce accidental data exposure but cannot support local USB administration. Data-capable cables are required for USB ADB and some provisioning workflows. Mark cable type visibly because identical connectors can have different capabilities.

Powered USB hubs should have their own adequate supplies. Avoid long chains of hubs; every tier adds connectors, failure points and potential bandwidth or enumeration problems. For large fleets, distribute devices across multiple host USB controllers and powered hubs rather than one massive tree.

- Use short, labeled cables.
- Maintain a spare cable bin by exact type.
- Log recurring disconnects by slot; intermittent cables cause disproportionate troubleshooting time.
- Use strain relief so the phone connector is not supporting cable weight.

Host enumeration

For Android, ADB identifies attached authorized devices by serial and transport. For Apple, management is normally handled through MDM for ongoing fleet operations, with Configurator or local tooling used for specific provisioning/service cases. [S3][S5]

Symptom

Likely class

Check first

Phone charges, no data
 Charge-only cable / port mode
 Known-good data cable
 Random disconnects
 Cable/hub/power instability
 Hub supply + logs
 Many devices disappear together
 Shared hub/controller fault
 Upstream hub and AC zone
 Slow media transfer
 Shared USB bandwidth
 Controller topology

10. Thermal engineering

Heat is a reliability and battery-lifetime problem. Apple documents a 0-35 C ambient operating range for iPhone/iPad and warns that high temperatures can permanently reduce battery capacity; charging may slow or stop when the device is too warm. [S11] Other manufacturers publish their own limits, which should be checked per model.

Sources of heat

Charging losses, display use, CPU/GPU work, radios and tightly packed devices all add heat. A rack near a window or inside a closed cabinet can run materially hotter than the room. Measure at phone inlet locations, not only at the HVAC thermostat.

Continuous 100% charge is also undesirable for battery aging. Where the manufacturer provides optimized charging or managed battery features, evaluate them. A fleet can be scheduled to maintain adequate charge without forcing every battery to remain at maximum state of charge continuously.

- Ambient intake temperature.
- Rack exhaust temperature.
- Representative battery/device temperature where accessible.
- Fan operation and dust loading.
- Thermal throttling or charging-paused events.

Airflow

Use a predictable front-to-back or bottom-to-top flow. Recirculation is the enemy: if hot exhaust immediately returns to the intake side, adding more fans can increase noise without improving temperature. Baffles or spacing can be more effective than raw fan count.

Design alerts well below absolute device limits so operators have time to react. The correct warning threshold depends on the device model and sensor being monitored.

Engineering checklist

- Baseline temperatures at 25%, 50% and 100% rack occupancy.
- Repeat test with all devices charging from low battery.
- Define warning and critical thresholds.
- Test a failed-fan scenario.
- Keep rack away from direct solar load.

11. Fire/electrical safety and UPS

A phone rack combines many lithium-ion batteries, charging electronics and AC connections. Use commercially rated electrical components, inspect cords and connectors, and follow local electrical/fire rules. OSHA guidance for extension cords emphasizes proper construction, grounding where required, strain relief and inspection; local regulations may impose additional requirements. [S31]

Safety principles

Do not treat consumer power strips as permanent high-density distribution without verifying their ratings and local code suitability. Do not defeat grounding or overcurrent protection. Remove swollen, damaged or unusually hot batteries from service immediately according to manufacturer and local hazardous-waste guidance.

Provide a clearly reachable main disconnect for the rack and an incident procedure for smoke, odor, battery swelling or abnormal heat. Staff should know not to puncture or compress a damaged lithium-ion battery.

- Rated PDU / distribution.
- Cable inspection schedule.
- No blocked ventilation around chargers.
- Smoke detection appropriate to the site.
- Fire response plan aligned to local authority guidance.

UPS strategy

Prioritize graceful operation rather than attempting to run all charging for hours. A practical UPS can keep router, firewall, switch, APs, controller and database online long enough to preserve state and shut down cleanly. Phones already contain batteries and may not need to be on UPS-backed charging circuits unless continuity requirements demand it.

Calculate UPS VA/W load from measured equipment and required runtime. Validate actual runtime with a controlled test after installation.

12. Host computers and local controllers

The local controller is the bridge between the physical fleet and the backend. It may host ADB, local media caching, telemetry collectors, a message broker edge client, a web dashboard, or Apple provisioning utilities.

Sizing

CPU requirements are modest for orchestration but can rise sharply if the host transcodes video, performs local AI analysis or handles many concurrent media transfers. RAM should cover the OS, device tooling, database/cache agents and headroom. Use NVMe storage if the host is a media staging point; otherwise keep large media in object storage or a dedicated NAS.

For a 10-25 device pilot, one reliable workstation or mini-PC may be enough. At higher scale, separate the fleet controller from heavy video processing and from the primary database to reduce blast radius.

- At least two network interfaces if you want clean management/device separation.
- Automatic startup of required services after reboot.
- Disk encryption.
- Admin accounts protected by MFA.
- Regular image/configuration backup.

Operating system choice

Android ADB works across Windows, macOS and Linux. Apple Configurator is a macOS tool, while long-term Apple fleet control should be performed with a suitable MDM rather than requiring every device to remain tethered to one Mac.

Choose the host OS your operators can patch and support reliably. Consistency beats novelty.

13. Inventory, labeling, spares and lifecycle

A fleet becomes manageable when physical identity and database identity are never ambiguous. Build inventory before scale.

Minimum asset record

Each device should have an internal asset ID, manufacturer/model, serial/IMEI where appropriate, purchase source/date, warranty status, OS version, storage, battery-health note, rack/slot, assigned creator or pool, management-enrollment status and retirement date. Sensitive identifiers should be access-controlled.

Do not encode creator names or other sensitive information directly into externally visible device labels. Use opaque asset IDs and resolve them in the management system.

- Asset barcode/QR.
- Cable and charger-port mapping.
- Spare status: ready / repair / quarantine / retired.
- Chain-of-custody when handed to an employee.

Lifecycle

Use a standard state machine: Received -> Inspected -> Enrolled -> Available -> Assigned -> Maintenance -> Quarantined -> Retired -> Wiped/Disposed. Every transition should be timestamped and attributable.

Maintain spare capacity. A fleet that operates at 100% device utilization has no room for OS updates, battery replacement or incident quarantine.

SECTION 3 - iOS Engineering: Identity, Provisioning, Display, Input and Media

Why iOS deserves its own section

The iOS problem is not simply "Apple is locked down." It is a set of deliberate platform boundaries that affect different tasks differently. Device management is strong; arbitrary UI injection is intentionally constrained. File transfer exists; Photos-library ingestion is a separate operation. A Mac can be a powerful provisioning/test host; that does not automatically mean it can deliver scalable arbitrary touch input without a phone-side component or attached hardware.

This section gives each mechanism a separate name so the design does not accidentally assume that one capability implies another.

3.1 Identity and reset model

IDFV is useful but not permanent

[VERIFIED-FIRST-PARTY] Apple's `identifierForVendor` is the same for apps from one vendor on one device and different for other vendors/devices. Apple documents that it changes after all apps from that vendor are deleted and later reinstalled. Therefore it is neither a global cross-vendor hardware serial nor a reliable forever-identifier.

DeviceCheck is persistent coarse state

[VERIFIED-FIRST-PARTY] DeviceCheck lets a developer store two bits and a timestamp associated with a device on Apple servers. Apple explicitly states that this state persists across app reinstall, device transfer between users, and Erase All Contents and Settings.

The design implication is subtle:

DeviceCheck does **not** expose a universal hardware serial.

The bits are developer-controlled state, not a descriptive fingerprint.

The state is useful specifically because ordinary install-local identifiers can change.

A reset therefore cannot be treated as a guaranteed clean slate from an app backend's perspective.

App Attest is installation-scoped

[VERIFIED-FIRST-PARTY] App Attest uses hardware-backed keys to prove app integrity, but the keys are unique per installation and do not survive reinstall, migration or restoration. Apple also states that attestations expose no hardware identifier.

[INFERENCE] DeviceCheck and App Attest should be mentally modeled as two different anti-abuse primitives: **persistent coarse memory** versus **current app-instance integrity**.

Used-device consequence

[INFERENCE] With any developer-controlled server-side state that survives ownership transfer, used hardware has unknowable historical state relative to a given app. That is an asset-risk consideration, not a reason to invent an "unban" procedure. For a legitimate fleet, record purchase source/IMEI/serial internally for inventory, validate hardware functionality, and treat unexplained platform-level device restriction through official support and asset-retirement processes.

3.2 Provisioning and supervision

Apple Configurator and device management solve **configuration**, not arbitrary third-party app control.

[VERIFIED-FIRST-PARTY] Apple Configurator Blueprints can apply apps, profiles and actions to multiple devices. Supervision unlocks additional restrictions/management features and is best established before activation or on a fully erased device. Unsupervising later erases the device.

This creates a day-zero choice for durable devices:

Choice

Benefit

Cost

Unsupervised

less intrusive setup; closer to consumer default

fewer management controls; more manual setup

Supervised

stronger configuration, restrictions, silent management options

must be planned before production state; reversing it erases device

[INFERENCE] For a five-device engineering pilot, supervision should be tested on **one** dedicated test phone before becoming the fleet default. The decision should be driven by which settings actually need automation, not by the assumption that "more MDM is always better."

3.3 Display paths

Path A - network mirroring

[INTERNAL-MEASURED] Xiaowei mirroring worked when the phone and Windows host were on the same local network. The observed path was network-based, not cable-based. This is a strong fact for that tested product/configuration.

Consequences:

local service discovery becomes part of availability;

VLAN segmentation can break discovery;

cellular-only devices cannot participate in a LAN-only display path;

a shared WLAN is a control-plane dependency even if Internet egress is different.

Path B - Apple's personal iPhone Mirroring feature

[VERIFIED-FIRST-PARTY] Apple's iPhone Mirroring requires a compatible Mac/iPhone, same Apple Account with 2FA, Bluetooth and Wi-Fi, and a locked nearby iPhone. It is a personal-device feature, not a documented rack-scale automation API. Do not build the fleet around it.

Path C - tethered iPhone screen capture on macOS

[OPEN] Dim's handover notes correctly point out that QuickTime can display a connected iPhone, and Apple AVFoundation provides programmable capture APIs for supported capture devices. The missing proof is whether a tethered iPhone's **screen stream** is exposed in a stable, public, multi-device programmable interface suitable for 5/10/20 concurrent phones.

A UI demonstration in QuickTime does not answer:

Does each phone appear as a separately addressable capture device/API object?

Is the stream hardware-accelerated or copied through a process with per-stream cost?

Can several streams be opened concurrently?

Is there an undocumented/private dependency that may change?

What USB bandwidth and CPU/RAM load occurs at scale?

Therefore TEST-MAC-CAPTURE-001 is a blocking experiment in Section 10.

3.4 Input paths

External HID/control board

[VENDOR-CLAIM] Niaozun and similar products claim to control iPhones with external hardware and vendor host software without jailbreak, developer signing or a custom app on the phone. Published product/API material describes pointer, swipe, keyboard, screenshot/vision and automation functions.

The correct engineering treatment is **adapter + containment + bench verification**:

control plane

|

v

vendor adapter service

|

v

vendor host API ----> control board ----> iPhone

Never let vendor API shapes leak into business logic. A future board should require one adapter replacement, not a backend rewrite.

WebDriverAgent / XCUITest / Appium

[VERIFIED-FIRST-PARTY/MAINTAINED-DOC] Appium's XCUITest driver requires WDA installed/provisioned on real devices, plus device trust, Developer Mode on current iOS and UI Automation. This is a mature QA path and gives semantic element-level automation.

It conflicts with a strict policy that nothing may be installed for control. That is a **policy trade**, not a technical impossibility.

Indirect input semantics

[VERIFIED-FIRST-PARTY] Apple's UIApplicationSupportsIndirectInputEvents documentation says that when indirect input support is enabled, pointing-device clicks produce UITouch.TouchType.indirectPointer; when disabled they are delivered as direct touches. If the key is absent, iOS 17+ defaults to indirect support and pre-iOS-17 systems default to the older direct-touch compatibility behavior.

This corrects two bad simplifications:

"Hardware pointer clicks always look exactly like a finger" - false at the UIKit API level on modern defaults.

"A platform definitely detects hardware control" - unsupported by the evidence. The OS exposes a distinguishable event; platform use is unknown.

[INFERENCE] Do not create production version-pinning around this. Run a controlled instrumented app on the exact device/OS/control-board combination to characterize event types for engineering knowledge, and treat third-party platform behavior as unknown.

3.5 The Lightning/USB coexistence question

[OPEN] The handover describes a practical concern: an accessory that requires the iPhone to act as USB host can conflict with the phone's simultaneous role as a USB device to a computer. Whether the inbound control board actually forces that topology is not established from the product claims.

This is a simple bench question with enormous architectural impact. If display/data to the host and external input can coexist through the intended board/hub arrangement, a wired pod becomes much simpler. If not, the system may need a network display channel or a different control route.

3.6 Media delivery

A byte copied to an iPhone is not automatically a Photos asset. The production requirement is not "file reached device"; it is "authorized asset is visible in the intended application's normal media picker with the expected orientation/date/type and can be removed/reconciled later."

A robust media state machine is:

APPROVED_IN_CLOUD

- > QUEUED_FOR_DEVICE
- > TRANSFER_STARTED
- > BYTES_PRESENT
- > PHOTO_ASSET_CONFIRMED
- > CONSUMED/POSTED (human or approved path)
- > RETENTION_ACTION

[VENDOR-CLAIM] If a control board exposes album/media endpoints, validate the *final Photos-library state* rather than trusting a 200 OK response.

[POLICY] Media provenance, creator authorization and content rights stay attached to the asset. The system may normalize filenames/transcoding/orientation for compatibility; it should not implement metadata fabrication intended to misrepresent origin.

3.7 Mac mini starting point

[VERIFIED-FIRST-PARTY] The current M4 Mac mini base configuration includes 16 GB unified memory and Thunderbolt/USB-C connectivity; memory can be configured higher.

[INFERENCE] For a lab expected to run Configurator, remote-access services, test tooling, multiple device processes and possibly several video/capture streams, **24 GB unified memory and at least 512 GB SSD** is the safer pilot purchase than the absolute base configuration. This is not because 16 GB is proven insufficient; it reduces the chance that the first scale test becomes memory-constrained. Do not jump to M4 Pro until TEST-CAPACITY-001 shows host CPU/I/O is the bottleneck rather than USB, vendor software or network mirroring.

3.8 iOS decision table

Need

Preferred first test

Why

Blocking unknown

Bulk setup

Configurator Blueprint

first-party, repeatable

exact settings coverage

Screen

Mac tethered capture experiment

could remove LAN display dependency

public scalable API not proven

Input without phone-side control app

inbound HID board

aligns with current policy

coexistence, event behavior, stability

Rich semantic automation

WDA/Appium

mature QA tooling

conflicts with no-install policy

Media import

board/Shortcuts/helper workflow test

must create real Photos asset

exact board behavior

Remote human ops

custom/GADS-like control interface

avoids video-of-video

WDA dependency for GADS iOS

Fleet management should make devices boring: repeatable enrollment, predictable settings, known application versions, controlled access and fast recovery. Apple and Android require different approaches.

24. Apple fleet architecture

Apple's enterprise deployment model centers on organization ownership, Automated Device Enrollment where available, supervision and Mobile Device Management (MDM). Apple Platform Deployment documents the lifecycle from planning and enrollment through configuration and ongoing management. [S5]

Figure 24-1. Simplified Apple MDM control flow.

Core components

Apple Business Manager (or the appropriate Apple organizational service) establishes organization ownership and can assign devices to an MDM service. Supervision provides additional management controls for organization-owned devices. The MDM service sends configuration profiles, app assignments and commands while devices communicate with Apple push infrastructure and the MDM service.

Use a commercial or internally operated MDM that explicitly supports the commands, restrictions and app deployment you require. The phone rack itself should not depend on one operator manually configuring every iPhone.

- Organization account and administrative roles.
- Device enrollment/assignment.
- Supervision.
- MDM server/service.
- APNs connectivity.
- Managed app distribution.

Design principle

MDM is the administrative plane. It is not a generic remote-control framework for simulating user gestures in third-party applications. Keep publishing workflows separate.

25. Supervision, enrollment and MDM

Supervision signals that an iPhone/iPad is organization-controlled and unlocks additional management capabilities. Automated Device Enrollment can supervise eligible organization-owned devices during setup; Apple Configurator can be used in certain manual preparation workflows. [S5]

Enrollment flow

At acquisition, record the serial and organization ownership. Assign the device to the intended MDM, boot/reset it, complete Automated Device Enrollment, verify supervision, apply baseline profiles, install managed applications, test connectivity, then mark the device Ready in inventory.

MDM commands typically use push notification to tell the device that a command is available; the device then contacts the management service over HTTPS, executes supported actions and reports results. [S6][S7]

- Wi-Fi/certificate profiles.
- Passcode and security policy.
- Managed apps.
- Restrictions appropriate to role.
- Lost mode/lock/wipe capability where supported.
- Inventory and compliance reporting.

Recovery

If a supervised device is lost or compromised, use the MDM's supported lock/wipe workflow and revoke relevant credentials. If it is reassigned, wipe and re-enroll rather than merely signing out of one social application.

26. Android Enterprise architecture

Android Enterprise provides management modes for personally owned and company-owned devices. For a phone farm, company-owned fully managed devices are normally the cleanest model because the organization can manage the complete work-only device. Dedicated devices are a subset that can be locked to a small application set. [S1]

Figure 26-1. Android Enterprise plus authorized ADB engineering.

Android Management API

Google's Android Management API uses Android Device Policy on the device to receive and enforce enterprise policies. Provisioning creates a device object bound to the enterprise. Enrollment tokens determine ownership/management mode; company-owned devices can use fully managed configurations. [S1]

Policies can control application installation, permissions, restrictions, networking settings, system updates and compliance behavior depending on Android version and management mode.

- Enterprise resource.
- Enrollment tokens.
- Policies.
- Device objects and state.
- Managed Google Play application policy.

Dedicated vs fully managed

Dedicated mode is suitable when the phone should expose only a controlled application set. If operators need several social apps, browser, camera, authenticator and internal tools, a normal fully managed configuration may be more practical than an overly restrictive kiosk.

27. Android provisioning and policy

Google documents QR, zero-touch, NFC and other provisioning methods. For company-owned work-only devices, full management allows the enterprise to manage all applications and enforce broad policy. [S1]

Pilot provisioning

For a small pilot, QR enrollment is straightforward: factory-reset the corporate device, enter the Android provisioning flow, scan the organization-generated enrollment QR, allow Android Device Policy to complete setup, then verify policy/application state. At scale, eligible zero-touch procurement reduces hands-on setup.

Enrollment tokens should be short-lived where practical and protected as administrative secrets because possession can allow a device to enroll into the enterprise configuration.

- Baseline Wi-Fi or setup network.
- Enrollment token handling.
- Policy assignment.
- Required apps.
- VPN client if required.
- Compliance validation.

Policy rings

Use Stable, Canary and Quarantine policies. Canary receives new OS/app/policy changes first. Stable is production. Quarantine removes unnecessary access while preserving enough management connectivity for investigation.

28. ADB for authorized engineering

Android Debug Bridge (ADB) is Google's command-line bridge between a development workstation and Android devices. It can communicate over USB and, on modern Android, paired wireless debugging. [S3] It is appropriate for provisioning support, logs, diagnostics and controlled testing on devices you own or are authorized to administer.

Architecture

ADB consists of a client, local server and device daemon. A workstation can enumerate authorized devices, open a shell, collect logs and transfer files. USB is usually the most predictable for a rack; wireless debugging reduces cables but depends on trusted Wi-Fi and pairing state.

Use a dedicated engineering account and workstation. Disable debugging on production devices if it is not needed, or restrict the workstation/network that can reach it.

- Inventory verification.
- Log collection.
- File push/pull for internal test assets.
- Package/version diagnostics.
- Network diagnostics.

Boundary

ADB can technically generate input events and interact with applications. This manual does not provide a workflow for using ADB to automate third-party social UIs where automation is prohibited or to make automated behavior appear human. Use supported APIs or human operation for those actions.

29. App distribution and configuration

A managed fleet needs predictable application versions. Install apps through MDM/managed app channels where available, define which apps are required, and avoid ad-hoc APK or IPA distribution unless you own the software and have an authorized enterprise development path.

Application catalog

Maintain a catalog with app name, package/bundle ID, platform, source, minimum version, required permissions, owner, data classification and update ring. Social apps, password manager, authenticator, VPN client, internal task app and device-health agent should each have an owner.

Permissions should be the minimum needed. For example, a publishing app may need photo-library access but not contacts. Review permissions after major app updates.

- Required vs optional.
- Version and update policy.
- Permissions.
- Data stored locally.
- SSO support.
- Uninstall/retirement behavior.

Internal companion app

Consider a small internal “operator companion” app that shows device ID, assigned creator/account, task queue, approved asset links, check-in status and incident button. It should not store social-account passwords and should not covertly drive other applications.

30. OS updates and maintenance rings

Ignoring OS updates creates security debt; updating every device immediately creates operational risk. Use rings.

Rings

Canary devices receive updates first and run representative workloads. After a defined observation window, a larger pilot ring updates, then production. Critical security fixes may justify accelerated rollout after compatibility testing.

MDM or Android Enterprise update controls can help schedule or defer updates within platform-supported limits. Do not leave devices indefinitely frozen on unsupported releases.

- Canary: 1-5% of fleet.
- Pilot: 10-20%.

- Production: remaining devices.
- Emergency: accelerated security rollout with explicit approval.

Maintenance window

Drain publishing tasks from the device, confirm battery/power, install update, reboot, verify management heartbeat, verify required apps and network, then return to Ready. Record the new build version automatically where possible.

31. Health telemetry and remote actions

A fleet dashboard should answer “what is healthy right now?” without requiring someone to walk down the rack.

Health model

At minimum track last heartbeat, OS/app version, storage free, battery/charging state, management compliance, network profile, last task, last error and current assignment. Temperature data availability varies by OS/device; use rack sensors even when device-level telemetry is limited.

Status should be derived, not subjective: Ready, Busy, Maintenance, Degraded, Quarantined, Offline.

- Heartbeat age.
- Charging state.
- Free storage.
- Wi-Fi/VPN state.
- Required app versions.
- MDM/Enterprise compliance.
- Task error rate.

Remote actions

Administrative actions can include lock, wipe, restart where supported, reapply policy, request inventory, rotate internal app tokens, or move a device to quarantine. High-impact actions should require stronger authorization and be written to the audit log.

SECTION 4 - Android Engineering: ADB, Enterprise Management and Integrity

4.1 Why Android remains operationally attractive

Android exposes a conventional engineering toolchain for organization-owned devices: ADB for development/debug control, Android Enterprise for managed-device policy, simpler media-file workflows and broad hardware choice. For a device lab, this usually lowers control cost and removes per-phone proprietary control hardware.

The mistake is turning that convenience into the claim "Android always resets cleanly." Modern Android anti-abuse design no longer supports that certainty.

4.2 Device identity: separate local IDs from backend memory

Modern Android restricts access to stable hardware identifiers for ordinary apps. That weakens old folklore about any app trivially reading IMEI/MAC/serial.

[VERIFIED-FIRST-PARTY] In 2026, Google's Play Integrity **device recall** is available in beta. A participating developer can store custom per-device state on Google's servers and retrieve it after reinstall or device reset.

[INFERENCE] The correct conceptual model matches iOS:

local identifiers may reset / be scoped / be restricted
 server-side device state can persist by platform-supported mechanism
 app integrity verdict evaluates current software/device trust

Therefore a reset is an administrative lifecycle operation, not an identity guarantee.

4.3 ADB and media workflows

ADB is valuable for owned test devices because it gives explicit device addressing, file transfer, shell commands and observability. It also makes media staging materially easier than iOS. The handbook uses ADB for legitimate engineering/QA tasks, not for evading application controls.

4.4 Android Enterprise

Use Android Enterprise/managed-device policy for configuration, app distribution, Wi-Fi/VPN policy, update rings and remote actions. Keep **management** distinct from **third-party UI control** just as on Apple: device policy is a fleet-admin function, not a substitute for an application's public API.

4.5 Integrity posture

For social/consumer apps that rely on platform integrity, use stock, supported, non-rooted hardware.

Root/custom-ROM/emulator techniques create support and integrity incompatibilities even in ordinary QA and should not be the production baseline.

4.6 iOS versus Android - decision lens

Dimension

iOS

Android

Consumer hardware consistency

high

varies by vendor/model

Organization management

strong

strong

Debug/control tool openness

constrained

ADB is powerful

UI automation on real devices

WDA/XCUITest signing path or attached hardware

broader ADB/testing options

Media staging

Photos ingestion is special

filesystem/media-store path simpler

Reset-persistent anti-abuse mechanism exists

DeviceCheck

Play Integrity device recall beta

Proprietary per-device control hardware required for strict no-install policy

possibly

generally not

[INFERENCE] For a durable, authorized multi-platform lab, Android should remain a genuine alternative rather than a cheaper imitation of the iOS plan. The correct choice depends on required target apps, device support horizon and operations cost, not on a myth that one OS is "unfingerprintable."

SECTION 5 - Networking: LAN, Wi-Fi, Cellular, NAT, DNS, Proxy and VPN

Networking is where “phone farm” discussions often become confused. Public IP address, local Wi-Fi address, DNS resolver, proxy, VPN tunnel, SIM carrier and device identity are different concepts. This part separates them and shows how to design reliable, auditable egress without using the network to misrepresent identity or defeat enforcement.

14. Network reference architecture

The recommended baseline is a business firewall/router, managed switching, one or more centrally managed access points, separate VLANs/SSIDs for fleet devices and management, and explicit DNS/egress policy.

Figure 14-1. Segmented network reference.

Zones

The management zone contains administrator workstations, MDM connectors, ADB hosts and monitoring. The device zone contains phones. The data zone contains database, media store and backups. A quarantine/guest zone can isolate new or suspect devices. ACLs should permit only the flows required between zones.

Segmentation reduces the impact of a compromised phone or stolen employee credential. It also makes troubleshooting easier because you know which traffic class a packet belongs to.

- Management VLAN.
- Device VLAN(s).
- Data/storage VLAN.
- Guest/quarantine VLAN.
- Optional VoIP/office user VLAN separate from the farm.

Internet edge

The firewall should log high-level connection and security events without collecting more personal data than necessary. Where a business VPN gateway or proxy is used, document which devices/apps use it and why.

Prefer deterministic configuration to ad-hoc per-phone settings. Configuration drift is one of the main reasons ten devices behave differently despite looking identical.

15. Wi-Fi radio and capacity planning

Phone density is a radio-capacity problem, not merely a “strong signal” problem. High-density wireless design requires enough airtime, channel planning and AP placement for the concurrent workload. Vendor high-density design guides emphasize planning client density and bandwidth rather than relying on maximum advertised AP client counts.

2.4 vs 5/6 GHz

2.4 GHz travels farther but has less clean spectrum and more interference. 5 GHz provides more channels and is usually preferable for dense phone racks when devices and region support it. 6 GHz can add capacity on compatible devices and APs but has different propagation and regulatory constraints.

Do not put a rack of many phones directly against one AP and assume proximity guarantees capacity. Very strong signals can coexist with contention. Use multiple APs only when channel planning supports it; simply adding APs on overlapping channels can make performance worse.

- Measure channel utilization and retry rates.

- Use wired Ethernet backhaul for APs.
- Disable legacy rates/features only after confirming device compatibility.
- Set reasonable transmit power rather than maximum everywhere.
- Perform throughput and latency tests from representative rack positions.

SSID design

Keep SSID count low because management overhead consumes airtime. Separate networks by security/policy need, not by creating one SSID per creator. VLAN assignment can provide segmentation behind a smaller number of SSIDs.

If using ADB wireless debugging, the workstation and device must meet the applicable pairing/network requirements. Android 11+ supports wireless debugging with pairing; use it on trusted networks only. [S3]

Metric

What it indicates

Typical response

Channel utilization

Airtime pressure

Reduce load / add properly planned capacity

Retry rate

Interference / weak link

RF survey, power/channel tuning

RSSI/SNR

Link quality

AP placement / obstacles

Latency/jitter

Congestion or WAN issue

Separate LAN vs WAN tests

Roaming events

AP handoff behavior

Tune only if devices actually move

16. Ethernet, switches and VLANs

All infrastructure that can be wired should generally be wired. APs, controller hosts, storage and servers should use Ethernet so wireless airtime is reserved for phones.

Switching

A managed switch allows VLAN trunks, per-port statistics, PoE power for APs, and port isolation where useful. Maintain a port map so a failed AP or controller can be found quickly.

For larger installations, redundant uplinks and a second switch may be justified, but complexity should follow actual uptime requirements.

- Label patch cables at both ends.
- Use separate management access for switches/APs.
- Back up switch configuration.
- Disable unused physical ports or place them in a restricted VLAN.

VLAN/ACL design

A VLAN is a Layer-2 segmentation mechanism; it is not security by itself. Inter-VLAN routing rules on the firewall or Layer-3 switch enforce which systems can communicate. A device VLAN typically needs DNS, NTP, internet access and access to explicitly approved internal services, not unrestricted access to the management network.

Block unsolicited device-to-device traffic unless the workflow requires it. This limits lateral movement if one handset is compromised.

17. IPv4, IPv6, NAT and CGNAT

Understanding address translation prevents mistaken assumptions about “one IP per phone.” A phone normally receives a private local IP address from DHCP. The router may translate many private IPv4 addresses into one or several public IPv4 addresses using NAT. A mobile carrier may perform another layer of carrier-grade NAT (CGNAT), meaning many subscribers share public IPv4 space.

IPv4/NAT

Private IPv4 addresses such as RFC1918 ranges are not globally routable. The edge router keeps connection state so return traffic is mapped back to the correct phone. NAT changes addressing; it does not inherently anonymize application identity.

Port/address translation also means a platform may see the same public egress IP for many legitimate devices behind a business connection. Conversely, the same phone may appear behind different public addresses as networks change.

CGNAT and mobile networks

CGNAT performs NAT inside the carrier network at large scale. This is common where public IPv4 addresses are scarce. IPv6 can provide globally routable addresses without the same NAT requirement, though firewalls still control inbound/outbound connectivity. [S18]

Do not design identity assumptions around IPv4 alone. Modern applications can use IPv4 and IPv6 depending on network and DNS responses.

- Record whether each segment has IPv6 enabled.
- Test application behavior over IPv4 and IPv6.
- When troubleshooting “wrong IP,” identify the local IP, router WAN IP and externally observed IP separately.

18. DNS and resolver design

DNS converts hostnames into addresses and is part of both reliability and privacy. Traditional DNS can be visible to the local network; DNS over HTTPS (DoH) and DNS over TLS (DoT) encrypt queries between client and resolver. [S17]

Resolver choices

A business can use the firewall as a forwarding resolver, an internal resolver, or a managed external resolver. The goal is consistent policy, fast failure diagnosis and appropriate logging. Do not mix several ad-hoc per-phone resolver settings unless there is a documented requirement.

If a VPN is used, decide whether DNS should also traverse the tunnel. Split DNS may be necessary for internal names. DNS leaks are not merely a “privacy hack” topic; mismatched routing can break corporate resources and make incident analysis confusing.

- Monitor resolver latency and failure rate.
- Use DNSSEC-validating resolvers where suitable.
- Keep internal hostnames out of public DNS.
- Minimize query logging retention to operational/security needs.

Failure diagnosis

Test in layers: can the phone reach the default gateway, can it reach a known IP, can it resolve a hostname, can it establish TLS to the target, and can the application authenticate? This avoids blaming “the proxy” for a DNS or certificate problem.

19. Forward proxies and proxy protocols

A forward proxy is an intermediary that makes outbound requests on behalf of clients. For HTTP(S), a proxy may forward requests directly or use the CONNECT method to create a TCP tunnel for TLS traffic. [S14] A proxy is different from a VPN: a proxy is usually application/protocol-aware, while a VPN normally routes IP packets through a tunnel.

Figure 19-1. Proxy, VPN, NAT and DNS are separate layers.

Why businesses use forward proxies

Legitimate uses include outbound policy enforcement, malware filtering, controlled egress, audit logging, caching in some environments, and allowing specific internal applications to reach the internet through a known gateway.

On mobile devices, proxy support varies by application and OS. A Wi-Fi proxy configured at the network level may not affect every protocol. A VPN-based egress design is often more comprehensive when the requirement is “all traffic from these apps/devices follows this path.”

- HTTP proxy: understands HTTP semantics.
- CONNECT: creates a tunnel commonly used for HTTPS through an HTTP proxy.
- SOCKS-style proxy: relays TCP and sometimes UDP without understanding HTTP content.
- Transparent/intercepting proxy: network redirects traffic; introduces certificate/privacy complexity for TLS inspection.

Proxy provenance and risk

Marketing terms such as “datacenter,” “residential” and “mobile” refer to how the public egress address is sourced. They do not create a compliance exemption. Third-party proxy providers introduce security, privacy,

availability and contractual risks because they may observe metadata, alter routes or lose access to address pools.

This manual does not recommend proxy rotation patterns or provider selection for avoiding platform detection. If an agency needs a proxy, choose it for an articulated business purpose such as security, regional testing with authorization, or controlled corporate egress, and perform legal/provider due diligence.

Engineering checklist

- Document why a proxy is needed.
- Confirm which apps/protocols actually use it.
- Test TLS/certificate behavior.
- Measure latency and failure rate.
- Define what the provider can log and retain.
- Create a rollback path to direct egress.

20. VPNs, tunnels and per-app routing

A VPN creates a logical network tunnel between device and gateway. On Android, VpnService creates a virtual TUN interface, allowing a VPN application to read outgoing IP packets and return incoming packets through the interface. Android supports always-on VPN and per-app allow/disallow lists. [S4]

Use cases

VPNs are appropriate for securing traffic over untrusted networks, reaching private corporate services, isolating a fleet behind managed gateways, and applying consistent egress/security policy. WireGuard is one modern protocol designed as a secure network tunnel; enterprise deployments may also use IPsec/IKEv2 or vendor clients. [S19]

Per-app routing can keep only managed applications inside a corporate tunnel while leaving OS update or other traffic direct, or it can do the reverse depending on policy. Test carefully because platform apps may depend on push services, CDNs and media endpoints outside a simple hostname list.

- Always-on mode for high-assurance corporate access.
- Kill/block mode if non-VPN traffic must not bypass policy.
- Per-app lists to scope tunnel use.
- Gateway redundancy for production.
- Key rotation and device revocation.

What a VPN does not do

A VPN changes the network path and may change the public IP address observed by a service. It does not change the account's history, app telemetry, device security state, content, user behavior or the platform's rules. Do not treat it as a method for falsifying identity or location.

21. Cellular, SIM and eSIM concepts

Cellular connectivity can provide redundancy or legitimate mobile testing, but it adds carrier contracts, SIM inventory, signal variation and CGNAT behavior. A mobile plan should be procured and used in accordance with carrier terms and local identity/registration laws.

Physical SIM vs eSIM

Physical SIMs are easy to move between compatible devices but create a physical inventory problem. eSIM can simplify provisioning on supported carriers/devices but may require carrier portals or activation workflows. In either case, map the subscription to the device and cost center in inventory.

Do not confuse the subscriber identity on a SIM with the identity of a social account. Keep telecom records and platform credentials in separate systems.

- Carrier name/plan.
- SIM/eSIM identifier stored with restricted access.
- Assigned device.
- Monthly data cap and roaming rules.
- APN/VPN requirements.
- Suspension/termination procedure.

Redundancy

For business continuity, a cellular router can provide WAN failover for the site. This is often easier to manage than giving every phone its own cellular plan. Individual phone cellular service should exist only where the workflow needs it.

22. Egress observability and troubleshooting

Operators need to answer: which path did this device use, which DNS resolver answered, what public IP was observed, did the VPN/proxy work, how much latency was introduced, and where did the request fail? Build observability for those questions without recording sensitive content unnecessarily.

Telemetry

Collect device ID, network profile, SSID, gateway reachability, resolver status, tunnel status, public egress test result, latency, packet loss and timestamp. Avoid storing credentials, message contents or unnecessary browsing history in network logs.

Use synthetic probes to your own endpoints for health checks. Do not continuously probe third-party platforms in a way that creates abusive load.

- DHCP lease and VLAN.
- VPN/proxy state.
- DNS response time.
- WAN packet loss.
- Gateway CPU/connection utilization.
- AP client count/channel utilization.

Troubleshooting ladder

Always isolate layers: device hardware -> Wi-Fi association -> DHCP -> gateway -> DNS -> tunnel/proxy -> TLS -> application authentication -> platform/API response. A documented ladder reduces random configuration changes that create new faults.

23. Network failure modes

Most fleet networking incidents are ordinary infrastructure failures: overloaded Wi-Fi, dead cables, DNS outages, expired tunnel credentials, saturated WAN links or misapplied ACLs.

Common scenarios

If all devices on one rack fail simultaneously, look for shared AP, switch, VLAN, DHCP or power dependencies. If only one application fails but other internet access works, look above the IP layer. If a subset of devices intermittently disconnects, inspect RF utilization and client-specific power-saving behavior.

When changing proxy/VPN configuration, deploy to a small ring first. A bad egress policy can strand the entire fleet from MDM or management services.

Resilience patterns

Dual WAN, redundant DNS resolvers, spare APs, configuration backups and staged changes provide more benefit than exotic networking. Keep a direct, secure management path so a failed content egress tunnel does not remove your ability to manage the device.

- [illegible]

Failure

Blast radius clue

First containment

AP failure

One RF cell/rack

Move clients / replace AP

DHCP failure

New joins fail

Restore DHCP / lease service

DNS failure

IP works, names fail

Fail over resolver

VPN gateway failure

Tunnel users only

Fail over or controlled bypass

WAN outage

Whole site

Secondary ISP/cellular failover

SECTION 6 - Client, Account and Credential Separation

6.1 The security objective

The goal is not to make authorized clients look unrelated to a platform. The goal is to prevent a mistake, breach, restriction, billing dispute, vendor failure or staff departure affecting clients who were not involved.

The internal account-separation reference ranks the most damaging agency failure modes as: shared business containers, credential spill, session bleed, shared billing, offboarding gaps, enforcement contagion and vendor concentration. Browser fingerprinting is real but lower in practical blast-radius priority.

6.2 Six independent separation layers

- 6. PLATFORM GRAPH ownership, partner/admin relationships, billing
- 5. IDENTITY/CREDENTIAL password, 2FA, recovery, business identity
- 4. BROWSER PROFILE cookies, storage, extension state
- 3. DEVICE/OS Windows/macOS user, filesystem, clipboard, backups
- 2. NETWORK IP, DNS, NAT, proxy/VPN, network operator
- 1. BEHAVIOR/CONTENT timing, assets, link patterns, writing style

A boundary at one layer does not cancel a relationship at another.

[INFERENCE] This is the most useful conceptual tool in the new research because it prevents "proxy thinking" from consuming the whole security design. If the same employee is explicitly an admin in two client Business Centers, a different IP does not erase the platform's own admin relationship. If two clients share one billing instrument, separate phones do not isolate payment risk.

6.3 Platform graph and delegated access

[VERIFIED-FIRST-PARTY] TikTok Business Center provides account/asset-level permissions for members and partners. This allows an agency to request scoped access rather than take custody of the client's primary credential.

[VERIFIED-FIRST-PARTY] X Delegate lets an owner grant admin/contributor access without sharing the owner's password. X explicitly notes that changing the account password does **not** remove delegates; revocation is a separate action.

For every platform, first ask: **does a first-party delegated/partner mechanism exist for the exact asset/action?** If yes, use it by default. If no, document why credential custody is unavoidable.

6.4 Client-owned containers

[POLICY] Where the platform supports business containers, the client owns the container and core asset. The agency receives scoped access. This means the client can revoke the agency without losing its asset, and an agency-level restriction does not automatically take ownership of the client's resources.

This is the difference between *operating an asset* and *owning the asset on the client's behalf*.

6.5 Credentials, 2FA and recovery

Credential custody should be minimized:

unique vault-generated password where password access is unavoidable;

no reuse across clients;

client-owned recovery route;

organization-managed 2FA with auditable access rather than one employee's personal phone;

staff never copy credentials into notes/chat;

rotation on offboarding/security event;

credential references in the control-plane database, never plaintext secrets.

[INFERENCE] A useful invariant is: **the database may know which secret object authorizes an action; it should not be able to print that secret.** Store a vault reference and authorization scope, not the password/token itself.

6.6 Browser profile is convenience isolation, not workstation security

[VERIFIED-FIRST-PARTY] Google documents that Chrome profiles keep bookmarks, history, passwords and settings separate. Google also warns that someone with access to the device can switch profiles. Chrome sign-in/sync can save passwords/history/tabs/settings to the associated Google Account.

Therefore:

one browser profile per client is useful for session hygiene;

separate OS users are stronger for local separation;

sensitive clients may justify separate workstations/VMs;

no personal Google Account sync in client profiles;

extension allowlists matter because a third-party extension becomes another data processor;

downloads/clipboard/screenshots/shared cloud folders must be included in the separation model.

6.7 Billing and vendor concentration

[POLICY] A billing dispute for Client A should be structurally unable to disable Client B. Use client-owned billing where possible; if the agency fronts spend, isolate instruments/accounting by client.

The same principle applies to schedulers, analytics, link shorteners and media tools. One convenience SaaS account that can access every client is a single breach domain. Document every vendor's client scope and revocation path.

6.8 Staff lifecycle

Every person gets an individual identity and least-privilege role. No generic shared "VA login." The joiner/mover/leaver process should be executable from an inventory:

person -> roles -> clients -> platform assets -> vault grants -> workstation/profile -> vendors

Offboarding is complete only when every edge is removed. Password rotation alone is insufficient where delegated access persists.

6.9 Incident containment

When one client is compromised/restricted:

stop automation/changes for that client only;

preserve logs and current state;

revoke/rotate relevant access;

verify no shared billing/vendor/session path affects other clients;

use official platform support/appeal routes as appropriate;

do not "test" other clients by repeating the suspicious behavior;

produce an incident review describing the actual linkage that created blast radius.

[INFERENCE] The correct success metric is **blast-radius boundedness**: one client incident remains one client incident.

SECTION 7 - Content Operations and Platform Behavior: Evidence vs Mythology

7.1 Why this section is separate from infrastructure

Infrastructure determines whether a device can be operated reliably. It does not prove how a recommendation system will rank content, whether an account has a hidden risk score, or what exact launch routine a platform prefers. Mixing these categories is how operator folklore becomes "technical fact."

7.2 The Instagram warmup evidence is contradictory by construction

The supplied industry review aggregated many creators/operator groups and records successful approaches ranging from day-one posting to multi-week warmup. It also records conflicting claims about aged accounts, proxies, anti-detect browsers and physical phones.

That source is valuable **because it preserves disagreement**.

[INFERENCE] If one operator can post immediately and another sees lower failure after delaying, then "number of warmup days" cannot be the only explanatory variable. Likely confounders include account provenance, content originality/quality, policy status, recommendation eligibility, audience fit, activity history, current abuse-enforcement pressure and operator behavior.

What can be operationalized safely

content rights and creator approval;
complete and accurate profile/business information;
consistent posting calendars that match a real content strategy;
quality/recommendation analytics;
stories/feed/reels as ordinary platform features used within policy;
A/B testing content formats on authorized accounts;
tracking changes over time without claiming an undocumented algorithm rule.

What should remain folklore/risk context

a universal 3/5/7/14-day "safe" warmup number;
claims that a proxy class guarantees account survival;
burn-and-replace account economics;
anti-detect/browser fingerprints as a magic fix;
"human-looking" automation designed to conceal automation;
ban-wave bypass instructions.

7.3 Content quality versus account age

A strong early content history can be correlated with better later performance without proving that "warmup" caused it. Treat content and account age as separate variables.

A good internal
experiment table looks
like:

Variable

Example controlled values

Keep constant

Account age

new vs established authorized account

creator, content class, audience target

Posting cadence

low/medium ordinary cadence

content quality and format

Format

photo/reel/story

account and daypart

Content set

historically strong vs exploratory

posting cadence

Link placement

approved platform-compliant destination patterns

content set

The point is not to discover a loophole; it is to stop attributing normal recommendation variance to whichever ritual happened before a successful post.

7.4 The five-post/fifth-story observation

[INTERNAL-MEASURED] XYZ has one owned Snapchat observation in which an under-optimized SFW account showed a significant traffic/subscriber change after its fifth public-profile story. Operator A independently described a preference for at least five profile items.

This is interesting convergence, but the correct conclusion is deliberately narrow:

it justifies a controlled hypothesis;

it does not prove an exact threshold;

it does not identify the mechanism;

timing/account aging/content differences remain confounders;

one account cannot estimate reliability or expected yield.

A legitimate experiment would compare a small number of authorized test accounts/content schedules with clear logging and stop if platform rules or account health indicate a problem. The result should be written as measured data with confidence intervals/limitations, not a production superstition.

7.5 Recommendation eligibility and sensitive content

Social platforms distinguish content that is allowed to exist from content that is eligible for broad recommendation. That distinction is especially important in adult-adjacent creator marketing: content can be non-violating yet receive restricted recommendation/discovery treatment.

[INFERENCE] For OFM-like operations, the creative/compliance system may have more impact on sustainable traffic than the phone hardware. The device farm cannot engineer around a recommendation-policy boundary. It can only make authorized operations more consistent and measurable.

7.6 Content pipeline as an engineering object

Treat each asset as a versioned object with:

creator/client owner;
 authorization/consent status;
 source/master checksum;
 content classification;
 allowed platforms/uses;
 derivative/variant lineage;
 approval state;
 expiry/revocation status;
 campaign/tracking metadata;
 device staging history.

The system should be able to answer: "Who approved this exact file, which accounts received it, when was it removed, and what performance metrics were associated with it?"

7.7 Attribution

Use first-party analytics, campaign parameters and owned redirect/landing-page analytics where permitted. Separate:

content impression/reach;
 profile visit;
 link click;
 landing-page session;
 downstream subscription/purchase where the platform and privacy policy permit attribution.

Do not pretend last-click proves causation. Retain channel/account/content identifiers so the team can compare performance without needing invasive cross-client tracking.

The technical fleet exists to support a controlled creator-content workflow. This part focuses on authorization, media, publishing, employees and measurement.

42. Creator onboarding and authorization

Before an account or device is assigned, the creator relationship should be documented and operationally represented in the system.

Onboarding packet

Record legal/business identity as required by the agency, contract reference, platform account ownership, recovery-channel ownership, allowed content categories, geographic/commercial restrictions, approved employees, revenue attribution identifiers and exit procedure. Sensitive identity documents require stricter access than routine content metadata.

The creator should know which agency systems and people will access which accounts. Authorization should be revocable.

- Contract/authorization.

- Account inventory.
- Recovery contacts.
- Content rights/consent.
- Brand rules.
- Escalation contacts.
- Offboarding instruction.

Technical onboarding

Create creator and account records, issue vault entries, assign least-privilege roles, import only approved media, configure tracking links, run one controlled publishing test and verify analytics before scaling.

43. Media ingest and rights metadata

Every media asset should enter through a controlled ingest point instead of being copied through random employee devices.

Figure 43-1. Rights-aware content pipeline.

Ingest

When an asset is received, calculate a checksum, store the original immutably, attach creator/campaign ownership, record source and rights scope, and create a QC status. The ingestion system should not silently strip or rewrite evidence relevant to rights and provenance.

If the agency performs edits, retain an edit lineage so the approved published variant can be traced to the source.

- Asset UUID.
- Checksum.
- Creator/campaign.
- Source.
- Rights/consent reference.
- Import timestamp.
- Classification.
- Retention/expiry.

Sensitive content

Adult material requires strict access control. Avoid unrestricted local synchronization. Use role-based access and watermarking for internal review only if it does not alter source evidence and the creator approves the workflow.

44. Content processing and variants

Content processing should optimize for platform-supported formats and campaign testing without deceptive manipulation.

Variant generation

Create explicit variants for aspect ratio, safe crops, subtitle/caption burn-in where authorized, cover frame, encoding bitrate and clip length. Store the transformation recipe and version ID so results are reproducible.

Use platform-published media specifications and test upload quality. Avoid “metadata spoofing” intended to make repeated content appear unrelated or to deceive integrity systems; that is different from legitimate transcoding or metadata minimization for privacy.

- 9:16 vertical variant.
- Platform-safe crop.
- Captioned variant.
- Thumbnail/cover.
- Source-preserving archival copy.

Quality control

Automated QC can detect resolution, duration, codec, audio presence, file corruption and duplicate checksum. Human QC should confirm brand, consent, visible private information and platform suitability.

45. Approval workflow

Approval is a gate, not a chat reaction. A task should prove which version was approved, by whom, for which account and time window.

Approval object

Approval should reference immutable asset version, creator/campaign, target platforms/accounts or a defined scope, approver identity, timestamp and expiration/revocation if applicable. A later edit creates a new version requiring new approval unless the policy explicitly allows that transformation.

High-risk or ambiguous content can require two-person review: creator/manager plus compliance/operations.

- Asset/version.
- Scope.
- Approver.
- Timestamp.
- Expiry.
- Notes/restrictions.

Revocation

If a creator withdraws authorization, the system should be able to identify queued tasks and previously published references associated with the revoked asset so operators can stop future use and execute contractual/platform removal procedures.

46. Publishing paths and official APIs

Publishing should use the most supportable path available: official API, approved scheduler/integration, or human-operated managed phone. The coordinator records all three the same way.

Official API path

TikTok’s Content Posting API allows registered applications to post content on behalf of authorized creators after the app obtains the required product configuration and scope; unaudited clients have restrictions. [S22] This demonstrates the preferred automation pattern: explicit creator authorization, documented API scope and platform-controlled rate/feature constraints.

For other platforms, verify current first-party developer documentation because availability depends on account type, application review and changing product policy.

- OAuth/delegated authorization where supported.

- Rate limiting.
- Idempotency.
- Post/status ID capture.
- Audit of API actor and creator authorization.

Human device path

If an action is mobile-only or API-inaccessible, create an operator task with approved asset, caption guidance and target account. The operator performs the action in the native app and records verification. The coordinator should assist the human, not secretly impersonate the human.

47. TikTok operational notes

TikTok is particularly explicit about integrity and automation. Its current guidelines allow authentic multi-account use but prohibit spam, fake engagement, circumvention and automation used to register or operate accounts in bulk. [S23]

Compliant engineering posture

Use the Content Posting API when the desired workflow is supported and the application can satisfy authorization/audit requirements. Otherwise use trained operators on authorized creator accounts and keep posting cadence/content strategy inside current platform rules.

Build analytics around content quality and conversion rather than account multiplication. A smaller number of healthy, authorized accounts with measurable attribution is easier to govern than a disposable-account model.

- Do not automate bulk account creation.
- Do not buy/sell or misrepresent account identity.
- Do not manufacture views, likes, comments or follows.
- Do not build location-hiding or enforcement-circumvention mechanisms.

Operational telemetry

Record post ID, publishing path, asset version, timestamp, operator/API application, outcome and available reach/click data. If a post is rejected or account restricted, preserve the platform notice and route it to policy review rather than automatically creating replacements.

48. Reddit, X and Snapchat operational notes

Each platform has its own rules, but authenticity and spam controls are common themes.

Reddit

Reddit requires authentic participation and prohibits spam/content manipulation. [S24] Subreddit-specific rules can be stricter than site-wide rules. Build a subreddit rule register if Reddit is part of the creator strategy; do not assume an action allowed in one community is allowed in another.

Where API access is used, comply with current Reddit Data API terms and rate/access conditions. [S25]

- Record subreddit/community rules and last review date.
- Avoid repetitive cross-posting that violates community spam rules.
- Respect NSFW labeling and content restrictions.

X

X's authenticity rules recognize legitimate third-party management but prohibit coordinated inauthentic behavior, duplicate/substantially similar amplification and ban evasion. [S27] This supports a model where the agency manages authorized creator accounts without creating fake networks to manipulate reach.

Snapchat

Snapchat's deceptive-practices policy prohibits spam, artificially inflated engagement, follower-growth schemes and other deceptive behavior. [S26] Native content workflows should therefore remain human or platform-supported rather than driven by hidden engagement bots.

49. Instagram/Meta operational notes

Meta's products have formal business/developer tooling as well as anti-spam and authenticity rules. Exact publishing capabilities and account requirements change, so the deployment should treat Meta integrations as versioned connectors that are validated against current first-party documentation before release.

Architecture approach

Prefer supported Meta business/account access and approved publishing integrations where available. Maintain separate account ownership, page/profile/business relationships and authorization records in the control plane.

Avoid building the system around large-scale repetitive posting, synthetic engagement or replacement-account churn. Even if a device can technically perform an action, the business should first ask whether the action is permitted and sustainable.

- Verify current Instagram API publishing requirements.
- Use delegated business roles where available.
- Capture post/media IDs for reconciliation.
- Monitor access-token expiry and permission changes.

Native app tasks

Stories, certain editing features or emerging formats may reach the native app before APIs. Route these to a human task and maintain evidence of the approved asset and target account.

50. OnlyFans boundary and account responsibility

OnlyFans is the monetization endpoint in many OFM funnels, but the social phone farm and the OnlyFans account should be treated as separate systems. Agency assistance does not remove the creator's legal or contractual responsibility for the account. Publicly archived terms and UK parliamentary evidence have historically emphasized creator control/responsibility; verify current OnlyFans Terms directly before implementation. [S28][S29]

Access

Do not assume that a social-media operations employee needs OnlyFans credentials. Separate traffic acquisition roles from subscriber/chat or account-administration roles. This reduces both privacy exposure and credential blast radius.

If third-party account assistance is permitted under current terms and creator contract, document the authority, employee scope, authentication method and revocation process.

- Creator retains required ownership/control.
- Agency access explicitly authorized.
- Separate roles and least privilege.
- MFA/recovery ownership documented.
- Offboarding and session revocation tested.

Data separation

Keep public social analytics, creator media and subscriber/private customer data in separate access domains. Do not copy subscriber conversations into a growth dashboard unless there is a documented lawful/business need and appropriate protection.

51. Tracking links and attribution

Without attribution, a phone farm can produce activity without knowing what generates revenue. Use campaign-aware links and first-party landing analytics where lawful.

Link structure

Create a campaign/link ID that resolves to the intended destination. Store creator, platform, account, campaign, asset and optional post reference. The redirect service records only necessary event data and should respect privacy/consent requirements.

UTM parameters are useful for general analytics, but an internal opaque link ID can be easier to rotate or update without exposing internal naming.

- Link ID.
- Creator/campaign.
- Source platform/account.
- Asset/post reference.
- Created/expired dates.
- Destination.

Attribution levels

Deterministic attribution uses a directly observed click and conversion identifier. Probabilistic or modeled attribution estimates influence when deterministic linking is unavailable. Label modeled results clearly; do not present them as exact facts.

Use cohort comparisons and revenue per campaign rather than optimizing solely for impressions.

52. Employee roles and access model

Employees are part of the system. Define roles so a new hire does not automatically receive every creator credential and media library.

Example roles

Fleet technician manages devices/network but does not need creator social passwords. Content operator sees assigned approved assets and target accounts. Growth manager sees analytics and campaign configuration. Security/admin manages access and incidents. Finance sees revenue/cost summaries but not raw sensitive content unless required.

Role-based access should be supplemented by creator/account scope. A content operator may be authorized for Creator A but not Creator B.

- Fleet technician.
- Content operator.
- Growth manager.
- Creator manager.
- Security/admin.
- Finance/analytics.

Joiner/ mover/leaver

Provision named accounts, train the employee, issue hardware/keys, review access when role changes, and revoke access immediately at departure. Rotate shared recovery secrets if any existed, and reconcile devices/keys physically returned.

Role

Devices

Creator media

Social credentials

Analytics

Fleet tech

Admin

No/minimal

No

Health only

Content operator

Assigned

Assigned only

Delegated/assigned

Task-level

Growth manager

View

Campaign

Usually no password

Broad

Security admin

Admin

As incident requires

Vault admin only

Audit

Finance

No

No

No

Revenue/cost

53. Daily/weekly operating cadence

A phone farm should be run as an operations function with handoffs, not as continuous improvisation.

Daily

Start with health: offline devices, failed backups, full storage, expired credentials, network alerts and platform notices. Then review the approved publishing queue. End the shift with unresolved incidents and blocked tasks explicitly handed over.

Avoid measuring employees purely by number of posts; that incentivizes spam and hides quality or conversion problems.

- Morning fleet health check.
- Approval/queue review.
- Execute tasks.
- Verify and reconcile post IDs.
- Review alerts/restrictions.
- Shift handoff.

Weekly

Report output, funnel metrics, account health, device/network reliability, policy incidents, top/bottom content, labor usage and next-week experiments. The report should say what is blocked and who owns resolution.

54. Quality control and audit

Quality control verifies content, account and task correctness; audit verifies who changed what.

QC sampling

Randomly sample completed tasks and compare the published post with the approved asset/caption/target. Verify links resolve correctly and attribution metadata is present. Sample failed tasks to ensure error classification is honest rather than simply retried until “success.”

Periodically inventory physical devices against the database and MDM/Android Enterprise records.

- Published asset matches approval.
- Correct creator/account.
- Correct tracking link.
- Post ID captured.
- No unauthorized local asset remains.
- Device assignment matches physical rack.

Audit log

Write immutable or append-only records for login, role changes, credential access, creator authorization changes, task approvals, high-impact device commands and deletions. Retain according to legal/business requirements, not indefinitely by default.

SECTION 8 - Control Plane and Remote Operations

8.1 Remote desktop is maintenance, not the product UI

Dim's handover correctly warns against "video of video": a remote user watching a Mac/PC desktop that itself shows 20 live mirrored phones compounds compression, bandwidth and latency.

[INFERENCE] The operator UI should therefore render **structured device cards**:

```
Device 07 | online | 31 C | 74% battery | USB ok | net ok
Account   | authorized asset reference
Task      | media-stage #884 | step 3/5
Preview   | screenshot age 2.1s [refresh]
Actions   | inspect | retry | quarantine | open live session
```

Live video is an escalation tool for one device, not the default dashboard for all devices.

8.2 RustDesk/GADS/custom roles

RustDesk/remote desktop: host maintenance, emergency manual work, software installation, OS troubleshooting.

GADS-like device farm UI: reservations, per-device screen/control, multi-user lab workflow. For iOS, current GADS approaches commonly depend on WDA; that collides with a strict no-install policy and must be evaluated explicitly.

Custom control plane: stable inventory/task/audit layer that can call vendor adapters or a WDA adapter without coupling business workflow to a single UI tool.

[INFERENCE] These are complementary layers, not mutually exclusive products.

8.3 macOS permissions and commissioning

Remote-control software on macOS depends on privacy/TCC permissions such as Screen Recording and Accessibility/Input Monitoring. Commission the host physically once, record the required grants, and create a recovery runbook for the case where an OS update/reinstall removes a grant.

The commissioning record should include:

```
macOS version/build;
remote-access client version;
which permissions were granted;
FileVault state/recovery method;
Remote Login state and firewall scope;
local break-glass admin identity;
hostname/static reservation;
time sync;
automatic restart after power failure if appropriate;
monitoring agent state.
```

8.4 FileVault without surrendering unattended recovery

[VERIFIED-FIRST-PARTY] Apple now documents that on Apple-silicon Macs with macOS 26 or later, FileVault can be unlocked over SSH after restart when Remote Login and network connectivity are available.

[INFERENCE] This changes the security tradeoff materially. The default should not be "turn FileVault off because the rack is remote." Instead, test the supported SSH-unlock workflow in the exact network/firewall environment, protect the credentials/keys, and maintain a physical break-glass procedure.

8.5 Node agent

Each host runs a node agent that:

- enumerates local devices;
- maps stable internal device UUIDs to transient USB/vendor IDs;
- reports heartbeat/resources;
- receives tasks for devices it owns;
- invokes a vendor-specific adapter;
- captures observations/screenshots/logs;
- enforces local concurrency/resource limits;
- survives coordinator interruption and reconciles afterward.

The coordinator never assumes a device exists because a database row says it exists. The node agent proves liveness.

The coordinator makes the phone farm a system rather than a pile of devices. It should model devices, creators, accounts, assets, approvals, tasks, results, networks and employees as separate entities with an auditable workflow.

32. Control-plane architecture

A practical control plane consists of an API service, relational database, object/media storage, task queue or broker, device/fleet connectors, authentication/RBAC, metrics/logging and an operator web interface.

Figure 32-1. Control plane and execution layers.

Service boundaries

The API service owns business rules. The relational database stores durable entities and task state. Object storage holds media and large artifacts. A broker or queue delivers asynchronous work. Device connectors translate generic management intents into Apple MDM, Android Enterprise, internal companion-app or other supported actions.

Keep platform publishing connectors separate from device-management connectors. This prevents a social API outage from breaking fleet administration and makes authorization boundaries clearer.

- API/auth service.
- PostgreSQL or equivalent relational store.
- Object storage.
- Queue/broker.
- Fleet connectors.
- Publishing connectors.
- Analytics/event pipeline.
- Web dashboard.

Deployment

For a pilot, these can run on one secured server using containers. For production, separate stateful services, backups and secrets. Avoid unnecessary microservices: operational complexity grows faster than device count.

33. Device-to-coordinator communications

There are two broad channels: enterprise management channels provided by Apple/Google, and a custom internal agent/companion channel controlled by the agency. Do not reinvent MDM for functions MDM already performs.

HTTPS model

An internal companion app can periodically call an HTTPS endpoint to register heartbeat, fetch permitted task metadata and report completion. Use short-lived OAuth-style tokens or device certificates rather than embedding a shared static API key in every phone.

For near-real-time updates, WebSocket or server-sent events can be added, but simple polling is often easier to operate at 10-100 devices. Poll intervals should use jitter so all devices do not hit the server simultaneously.

- TLS for all remote traffic.
- Unique device identity.
- Token rotation/revocation.
- Idempotent task receipt.
- Clock synchronization.
- Backoff on failure.

Message broker

MQTT is a lightweight publish/subscribe protocol widely used for constrained devices and IoT-style messaging. [S20] A companion app could publish health telemetry and subscribe to a narrow command topic, but do not expose a broker directly to the public internet without strong authentication, authorization and TLS.

A queue/broker is transport, not the source of truth. Durable task state belongs in the database.

34. Task queue and state machine

Every action should move through explicit states so operators know whether it is waiting, running, verified or failed. Silent retries hide problems and can create duplicate posts.

Figure 34-1. Auditable task lifecycle.

Reference states

Draft: not yet approved. Approved: content/action permitted. Queued: eligible for scheduling. Assigned: bound to executor/device/operator. Executing: in progress. Verified: expected result observed. Closed: complete and immutable except for annotations. Failed tasks enter a review/retry branch with a reason.

Use idempotency keys for API publishing so a network retry does not create a second post. Human tasks should require the operator to record the platform post ID or a verification artifact before closure.

- Maximum retry count.
- Retryable vs terminal error classes.
- Deadline/SLA.
- Priority.
- Executor type.

- Required approval IDs.
- Verification method.

Scheduling

Scheduler logic should consider platform/account restrictions, operator shift, device readiness, campaign windows and content approvals. It should not create prohibited spam by blindly maximizing throughput.

35. Database and entity model

The database should model real business entities instead of embedding everything in a spreadsheet row. This supports permissions, history and reporting.

Figure 35-1. Core entity separation.

Core entities

Creator represents the contracted person/business. Account represents one platform account and stores ownership/authorization metadata, not plaintext credentials. Device represents a physical asset. NetworkProfile describes approved egress configuration. MediaAsset stores immutable asset metadata and storage URI. Approval links a person to a permitted action/asset. Task describes work. Event/Metric stores observed outcomes.

Use stable UUIDs internally. Human-readable labels can change; primary identity should not.

- Creator.
- PlatformAccount.
- Device.
- NetworkProfile.
- MediaAsset / AssetVersion.
- Approval.
- Task / TaskAttempt.
- Operator.
- AuditEvent.
- MetricEvent.

History

Use append-only audit records for security-relevant changes. For mutable business records, store `updated_at` and `updated_by`; for especially sensitive fields keep version history or event-sourced change records.

36. Object/media storage

Creator media is valuable and sensitive. Centralize it in access-controlled object storage rather than uncontrolled phone camera rolls and employee downloads.

Storage model

Use immutable source assets plus derived versions. Store SHA-256 or another strong checksum for integrity, creator ID, usage authorization, source, creation/import timestamp, classification and retention/expiration. A derived version should point back to its source and transformation recipe.

Phones should download only the assets required for assigned tasks, ideally through time-limited signed links. Clear local staging areas on reassignment/offboarding where technically possible.

- Server-side encryption.

- Restricted buckets/containers.
- Versioning where useful.
- Lifecycle/retention rules.
- Backup/replication.
- Access logging.

Content safety

Only store adult creator content for verified adults with documented authorization and applicable consent/releases. Build a takedown/revocation process that can locate every derived copy from an asset ID.

37. API design

The internal API should expose business intentions, not raw device-control primitives. “Assign approved asset X to account Y” is safer than “tap coordinate 400,900.”

Resource examples

Use REST or another consistent RPC model with resources such as /devices, /accounts, /assets, /approvals, /tasks, /events and /metrics. Every write should authenticate the caller and enforce role/creator scope.

Use request IDs and idempotency keys. Return machine-readable error codes. Record actor, source IP/device, target object and result in audit logs for security-relevant writes.

- GET /devices/{id}.
- POST /tasks.
- POST /tasks/{id}/approve.
- POST /tasks/{id}/assign.
- POST /tasks/{id}/result.
- GET /accounts/{id}/metrics.

Versioning

Version the API contract before external clients depend on it. Internal rapid changes are easier if the companion app and backend can tolerate one previous schema version during rolling updates.

38. Messaging and event transport

Asynchronous messaging decouples slow device/network actions from interactive web requests. A user should not keep an HTTP request open while waiting for a phone task to finish.

Queue semantics

At-least-once delivery is common and requires idempotent consumers. Exactly-once behavior is usually implemented at the application level using unique keys and database constraints rather than assumed from the broker.

Use separate queues/topics for device commands, publishing jobs, analytics ingestion and low-priority maintenance so one workload cannot starve another.

- Dead-letter queue.
- Retry delay/backoff.
- Maximum age.
- Consumer concurrency.
- Per-account/device serialization where necessary.

MQTT topic example

For telemetry, a namespace such as `fleet/{device_id}/health` and `fleet/{device_id}/events` is easy to authorize. Commands should be narrow and typed. Do not create a universal remote-shell topic in a production broker.

39. Metrics, logs and traces

Prometheus is a common metrics and alerting system built around time-series data and label dimensions. [S21] Whether using Prometheus or another stack, distinguish metrics, logs and traces.

Metrics

Metrics answer aggregate questions: online device count, task success ratio, queue depth, API latency, Wi-Fi loss, VPN tunnel availability. Keep label cardinality controlled; do not use a unique post URL as a Prometheus label if it creates millions of series.

Logs answer event details: device enrolled, task failed, credential reference rotated. Traces connect calls across services when debugging distributed workflows.

- Fleet online %.
- Task success/failure by executor.
- Queue age P50/P95.
- Publishing connector error rate.
- Database latency.
- Storage capacity.
- Network health.

Alerting

Alerts should be actionable: “>20% of devices in rack B missed three heartbeats” is useful; “CPU 71%” may not be. Add runbook links to alerts.

40. Credential and secret handling

Credentials are one of the highest-value assets in an OFM system. Do not store platform passwords, session cookies, MFA seeds or API keys in source code, shared spreadsheets, task descriptions or chat channels.

Vault model

Use a password/secret manager that supports per-user access, audit logs, MFA and revocation. The application database stores a reference such as `vault://creator/123/platform/x` rather than the secret value. Backend services receive only the secret they need, ideally for a limited time.

Where a platform supports delegated access, teams/business accounts or OAuth, prefer that over sharing the creator’s primary password.

- Unique operator accounts.
- MFA/hardware keys for administrators.
- No shared root credential.
- Recovery codes stored securely.
- Immediate revocation on offboarding.
- Periodic access review.

Session management

Social apps may maintain long-lived sessions on phones. Treat the device itself as a credential-bearing asset: lock it, encrypt it, inventory it, and wipe/revoke on loss or reassignment.

41. Availability, backup and disaster recovery

The control plane should survive ordinary failures without losing authorization records, task history or media.

Backup scope

Back up relational databases, configuration, audit records, object-storage metadata and essential media according to business/legal requirements. Secrets should be backed up through the vault's supported recovery mechanism, not exported into general backups.

Test restore. A backup that has never been restored is an assumption, not a recovery plan.

- Define RPO: acceptable data loss window.
- Define RTO: acceptable service recovery time.
- Keep at least one backup copy isolated from primary credentials.
- Document dependency order: identity -> database -> object store -> queue -> API -> workers.

Degraded operation

If the coordinator is down, operators should have a clear rule: pause new publishing or use an approved manual fallback with later reconciliation. Do not create a shadow spreadsheet that permanently bypasses audit and approval controls.

SECTION 9 - Observability, Reliability, Security and Incident Response

The fleet contains devices, credentials and sensitive creator content. Security therefore has to be designed into networking, software and employee workflows.

55. Threat model

A threat model identifies assets, adversaries, attack paths and controls. The most realistic threats are often credential theft, malicious/curious insiders, lost devices, vulnerable phones, exposed admin panels, leaked media and configuration mistakes.

Assets

Creator accounts, social sessions, content, legal authorization records, subscriber/private data where applicable, analytics/revenue, device inventory, network credentials and administrative secrets.

Rank assets by impact and limit who can access them. A fleet technician does not need subscriber data; a growth analyst does not need MDM wipe rights.

- External credential phishing.
- Malware on admin workstation.
- Stolen/unreturned phone.
- Insider media exfiltration.
- Misconfigured public storage.
- Compromised VPN/proxy account.
- Unpatched mobile OS.

Controls

MFA, vaulting, MDM, encryption, segmentation, EDR on admin hosts, patching, least privilege, secure backups, audit logs and employee offboarding cover most high-probability paths better than exotic anti-fraud tricks.

56. Platform restriction and enforcement handling

When a platform restricts a post or account, treat it as a policy/operations incident, not an automatic trigger to create replacement identities.

Workflow

Preserve the notice, account/post ID, timestamp, asset, publishing path and recent changes. Determine whether it is content policy, spam/integrity, security login, copyright, age/consent or an unknown issue. Use the platform's appeal/remediation process where appropriate.

Pause similar queued tasks if the issue may repeat. A single restriction can reveal a broken campaign rule that would otherwise affect dozens of accounts.

- Capture evidence.
- Classify cause.
- Contain similar tasks.
- Appeal/remediate legitimately.
- Update campaign/policy rule.
- Document decision.

Prohibited response pattern

Do not make “ban evasion” a software feature. Automated creation of replacement accounts, identity/fingerprint spoofing, location falsification or rapid proxy/device churn can violate platform rules and create fraud/security risk. TikTok and X explicitly address circumvention/ban evasion in their authenticity rules. [S23][S27]

57. Incident response

Incident response should be predefined because the first minutes of a credential leak or overheating event are not the time to invent authority and evidence rules.

Figure 57-1. Incident-response sequence.

Seven stages

Detect -> contain -> preserve evidence -> assess scope -> remediate -> recover -> review. For a lost phone, containment may include MDM lock/wipe and secret revocation. For overheating, containment is physical power isolation and safe handling. For a content leak, containment includes access revocation and distribution/takedown actions consistent with law and platform processes.

Maintain an incident commander role so multiple employees do not issue conflicting changes.

- Incident ID and severity.
- Affected creator/account/device.
- Timeline.
- Actions/actors.
- Evidence.
- External notifications if required.
- Root cause and corrective action.

Testing

Run tabletop exercises for: lost device, compromised operator credential, database outage, storage exposure, creator revokes access, network failure during publishing, and rack thermal alarm.

58. Privacy and data minimization

A growth/fleet system should not become an uncontrolled surveillance database. Collect the minimum data needed to operate and secure the business.

Minimize

Device health does not require reading personal messages. Network health does not require storing full destination histories indefinitely. Analytics often need campaign/link IDs rather than raw IP addresses. Separate security logs from marketing data and apply different retention policies.

If employees operate creator accounts, document monitoring and privacy expectations in employment/contract policy and comply with applicable labor/privacy law.

- Purpose for each field.
- Retention period.
- Access role.
- Encryption requirement.
- Deletion/export process.

Data subject/creator exit

Be able to locate creator-specific records and assets for contractual deletion/return while preserving records the agency is legally required to retain. Avoid schemas that mingle multiple creators' sensitive content in untraceable folders.

59. Business continuity and creator exit

Continuity is not only infrastructure uptime; it includes a clean way for a creator to leave without the business or creator losing control of accounts.

Creator exit

Freeze new tasks, inventory pending posts, return/transfer authorized credentials or delegated roles according to contract/platform capabilities, revoke agency sessions, remove employee access, stop scheduled content, export agreed analytics/assets, and wipe creator material from reassigned devices.

Confirm recovery email/phone and MFA ownership before disabling agency access so the creator is not accidentally locked out.

- Stop queue.
- Reconcile scheduled/published content.
- Transfer/revoke access.
- Export/return agreed assets.
- Delete/wipe according to policy.
- Record completion.

Continuity

Document who can operate if the primary fleet admin is unavailable. Store network/MDM recovery procedures securely. At least two appropriately authorized people should be able to recover the system without sharing one administrator password.

SECTION 10 - Experiments, Pilot Build and Scaling Gates

10.1 Experiment discipline

The project has enough contradictory research that architecture-by-opinion would be expensive. Every experiment should have:

question - one decision it resolves;

setup - exact device/model/OS/host/hub/software versions;

procedure - reproducible benign steps;

measurements - logs, timestamps, power, CPU/RAM, packet/USB observations as relevant;

pass/fail - defined before the test;

artifacts - screenshots/logs/results stored in the repo;

decision unlocked - which ADR/procurement choice may now be made.

10.2 Gate 0 - inventory before engineering

Record what already exists before buying alternatives:

every phone model/OS/battery health/port state;

chassis/hub make, upstream link, PSU rating;

Windows host specs;

available Mac mini candidate;

Xiaowei version/configuration and exact successful steps;

inbound Niaozen SKU/order/doc/API package;

cables/adapters/power supplies;

network/AP/router model and available VLAN/policy-routing features.

The fastest way to waste money is to design from a vendor listing while the actual bench already disproves it.

10.3 TEST-MAC-CAPTURE-001 - does the proposed Mac USB display path exist programmatically?

Question: Can one or more tethered iPhones be addressed programmatically as screen/video sources without WDA, AirPlay/network mirroring or a phone-side control app?

Method: use owned test iPhones and public Apple-supported APIs/tools only. Enumerate capture sources, open streams if exposed, record resolution/fps/latency/CPU/RAM/USB behavior. Repeat at 1, 2, 5 devices before extrapolating.

Pass: a documented, repeatable programmatic stream for each connected phone with stable identity and no LAN dependency.

Fail: only GUI QuickTime capture; no separately addressable public stream; instability beyond one phone; hidden network dependency.

Decision: whether Mac USB display can replace network mirroring.

10.4 TEST-BOARD-001 - inbound control board capability

Validate only the purchased unit before ordering more:

enumerate/control through existing hub;

API authentication and device identification;

click/swipe/text on a test screen;
 reconnect after cable/host restart;
 one script repeated 20 consecutive cycles;
 vendor process network connections captured for supply-chain review;
 CPU/RAM usage and crash behavior;
 whether the board requires/changes USB topology.
 Do not connect production client accounts during this test.

10.5 TEST-IOS-INPUT-001 - characterize event type in an owned test app

Build a tiny internal UIKit test app that logs `UITouch.TouchType` and gesture event classes. Compare finger versus the control board on each relevant iOS version. This is a platform-engineering measurement, not a third-party detection test.

Pass: event behavior is documented and reproducible.

Decision: understand compatibility risk for apps/UI frameworks; do not infer enforcement behavior.

10.6 TEST-MEDIA-IOS-001 - real Photos ingestion

Send a known test JPEG/video through the board's claimed file/media route. Verify:
 asset visible in Photos;
 visible to a normal image picker;
 correct orientation/type/duration;
 duplicate behavior;
 deletion/reconciliation;
 batch behavior;
 behavior after restart.

A successful filesystem copy with no Photos asset is a fail.

10.7 TEST-COEXIST-001 - display and input simultaneously

If Mac USB capture works and the board works, connect them in the intended final topology. Confirm the phone stays visible to the host while accepting external input. This settles the "USB device versus accessory host" concern empirically for the actual board.

10.8 TEST-CAPACITY-001 - pod scaling curve

Run the **real workload**, not idle connected phones, at 1/5/10/20 devices. Measure:
 host CPU/GPU/RAM;
 USB errors/enumeration loss;
 network throughput/mDNS/discovery if used;
 screenshot/display latency;
 task success/latency;
 power draw/temperature;
 remote-operator responsiveness.

Stop adding devices when reliability or recovery becomes nonlinear. That point, not the hub's printed port count, defines pod size.

10.9 TEST-THERMAL-001 - soak

Run at expected brightness/charge/control workload for at least one business-day window, then longer. Record per-device/rack temperature and power. Inspect batteries/ports. Test fan failure if safe and monitored.

10.10 TEST-REMOTE-001 - unattended recovery

From a remote location, verify:

host reboot;

FileVault recovery method if enabled;

remote-access service start;

node agent start/reconciliation;

device re-enumeration;

one failed device quarantined without losing the pod;

permissions failure detection.

10.11 Scale gates

Gate 1 - 1-2 device evidence bench

All architecture-blocking unknowns measured. No volume purchase.

Gate 2 - 5-device pilot pod

Inventory/control plane running; authorized test content; independent health; remote recovery; audit logs.

Gate 3 - 10-device stable pod

Seven-day operational soak; no unexplained enumeration loss; known failure/retry behavior; operator can run it without Dim physically present.

Gate 4 - 20-device production pod

Only if capacity data supports it. Power/thermal margin proven; remote operations usable; incidents bounded; vendor adapter stable.

Gate 5 - multi-pod

Scale by duplication. The central control plane adds pods; it does not increase one pod's mystery density.

10.12 Purchase rule

[INFERENCE] Never purchase 20 proprietary control boards because one vendor demo looked good. The minimum procurement evidence is: one board, exact hub, exact phone/OS, exact host, 20 unattended cycles, media/control path confirmed, supply-chain containment plan documented, and an adapter interface implemented.

Scale in stages. A ten-device pilot should prove enrollment, physical design, network stability, queueing, attribution and incident recovery before the agency buys fifty more phones.

60. Scaling from 5 to 100+ devices

Scaling changes the bottleneck. At five devices, manual labeling and one AP may be adequate. At fifty, power zones, Wi-Fi capacity, management automation and staffing dominate. At one hundred, redundancy, change control and inventory discipline become mandatory.

Stage 1: 5-10

Goal: validate workflow. Use complete standard phones, one or two device models, managed Wi-Fi, a small controller and basic MDM/Android Enterprise. Measure actual power, heat, posting labor and conversion.

Do not optimize rack density yet. Optimize observability.

Stage 2: 25-50

Add segmented VLANs, multiple planned AP cells as needed, rack zones, UPS-backed network/control plane, automated enrollment, formal content approvals, task queue, metrics and scheduled maintenance. Separate backend/database from heavy media processing if necessary.

Stage 3: 100+

Use multiple physical racks/zones, standardized spares, automated inventory reconciliation, redundant network edge, tested disaster recovery, role-separated staff and canary change rings. Consider multiple sites only after one site is operationally mature.

Dimension**5-10****25-50****100+**

Enrollment

Manual assisted

MDM/Enterprise standardized

Automated procurement/enrollment

Power

Single measured zone

Multiple protected zones

Rack/site capacity plan

Wi-Fi

1 planned AP may suffice

Capacity-driven multi-AP

RF design + monitoring

Controller

Single host

Separated roles as needed

HA services

Staffing

1-2 cross-functional

Defined roles

Shift + incident ownership

61. Cost model and BOM categories

Total cost is CAPEX plus recurring OPEX. A cheap phone can be expensive if it requires twice the labor or fails frequently.

CAPEX

Phones, rack/enclosure, chargers/PD supplies, cables, PDU, UPS, router/firewall, managed switch, APs, controller/workstations, environmental sensors, spare devices and installation materials.

Price bands vary strongly by country and used-device market. Obtain current local quotes rather than relying on a static web price in this manual.

- Devices.
- Rack/trays.
- Power distribution.
- Network.
- Controller/storage.
- Spares.
- Safety/environmental monitoring.

OPEX

Internet/cellular plans, MDM licenses, cloud hosting, object storage/egress, monitoring, domain/link infrastructure, employee labor, repairs/batteries, platform/developer costs where applicable, legal/compliance and backup storage.

Allocate shared infrastructure by campaign or creator using a transparent model. Labor is often the largest cost in a device-first workflow.

Cost class

Driver

Track as

Phone hardware

Device count/lifecycle

Monthly amortization

Power/network

Rack/site
 Shared infrastructure
 Software/MDM
 Managed devices/users
 Recurring license
 Cloud/storage
 GB, requests, compute
 Usage
 Connectivity
 WAN/SIM plans
 Monthly recurring
 Labor
 Tasks/hours
 Direct operating cost

62. Pilot build - 10 device reference

The reference pilot is deliberately conservative. Its goal is to prove the system and gather measurements, not maximize density.

Hardware categories

Ten organization-owned phones from one or two supported model families; a ventilated non-compressing rack/tray; reputable multi-port charging or zoned chargers; labeled data cables for the subset needing USB admin; business firewall/router; managed PoE switch; one appropriately placed business AP; controller workstation; small UPS for network/controller; temperature sensor; spare phone and cables.

Do not purchase dedicated proxy infrastructure for the pilot unless there is a legitimate documented networking requirement. Direct business internet with proper segmentation is simpler to debug.

Software

MDM/Android Enterprise, inventory database, credential vault, basic coordinator/task board, object storage, tracking-link service, metrics/logging and a weekly report. TikTok or other official APIs can be integrated only after platform application authorization requirements are understood.

Pilot exit criteria

Every device can be enrolled/recovered, every account is authorized, every task is attributable, network/thermal behavior is stable, backup restore works, creator exit can be executed, and the business can calculate cost per successful campaign output.

Engineering checklist

- Run 72-hour full-load thermal/network test.
- Simulate one AP/router/controller restart.
- Wipe and re-enroll one device.
- Revoke one operator and verify access is gone.
- Publish/verify through every supported path.
- Restore database from backup.

63. 25-50 device expansion

Expansion is justified only after pilot measurements show stable operations and positive business value.

Physical expansion

Divide the rack into power/service zones, add airflow based on measured heat, add AP capacity based on measured airtime/utilization, and maintain at least 5-10% spare device capacity. Use port/slot maps and environmental alerting.

If a single controller still performs ADB, video transcoding, database and dashboard functions, split those roles before they become a single point of performance and failure.

Process expansion

Introduce formal change management: canary devices, scheduled OS/app updates, weekly access review for high-privilege roles, incident severity levels and capacity dashboards. Add a second trained administrator before the system becomes dependent on one person.

Software expansion

Move task execution to asynchronous queues, implement idempotency, add metrics/alerting, automate inventory reconciliation and make the creator/account authorization model explicit in every API request.

64. 30/60/90 day implementation roadmap

The roadmap assumes the agency starts with no mature device-management control plane. Adjust based on existing infrastructure.

Days 0-30 - prove the foundation

Inventory creators/accounts; define policy boundary; select 5-10 devices; deploy network segmentation; establish MDM/Android Enterprise; create rack/power/thermal pilot; deploy credential vault; implement creator/account/device/asset database; create manual task queue and tracking links; record baseline costs.

Deliverables: physical pilot, authorization register, network diagram, enrollment SOP, incident contact list, first weekly funnel report.

Days 31-60 - automate administration

Build coordinator API and companion/dashboard; add task state machine, approvals, object storage, health telemetry, metrics/alerts, canary updates and backup restore tests. Integrate one supported publishing API such as TikTok Content Posting API if business requirements and app authorization permit.

Deliverables: auditable task flow, fleet dashboard, monitoring, restore evidence, controlled API publishing path.

Days 61-90 - scale and govern

Expand to 25-50 only if KPIs justify it. Add power zones/AP capacity, formal roles, weekly access reviews, incident exercises, creator exit test, cost allocation, performance experiment process and quarterly platform-policy review.

Deliverables: scaled rack, runbooks, staff training, policy register, cost/conversion dashboard and go/no-go criteria for further scale.

65. Acceptance test plan

The installation is not complete because the phones turn on. Acceptance tests prove the physical, network, management, security and business workflows.

Physical/electrical

Measure AC load at idle and representative peak, verify no supply exceeds its rating, confirm rack temperatures in normal and degraded fan conditions, inspect cable strain and confirm emergency disconnect. Run a controlled power failure and verify UPS behavior for network/control systems.

Network/fleet

Verify DHCP/DNS/VLAN policy, Wi-Fi stability, tunnel/proxy behavior if configured, management reachability, enrollment, remote lock/wipe or equivalent supported actions, OS/app inventory and recovery after reboot.

Business/security

Create an approved asset, assign a task, publish via each supported path, capture post ID, verify tracked link, ingest analytics, produce weekly report, revoke an operator, remove a creator, restore a backup and execute one incident tabletop.

Engineering checklist

- All test evidence attached to acceptance record.
- Known exceptions have owner and due date.
- No production scaling before critical failures are closed.

SECTION 11 - Claude / Engineering Execution Specification

11.1 What Claude should do first

The coding agent should **not** begin with social-platform automation. It should begin with the neutral fleet substrate that is testable without any client account:

repository and configuration conventions;
 device inventory database;
 host/node agent heartbeat;
 vendor adapter interface with a fake/simulator adapter;
 task state machine;
 observation/log model;
 operator dashboard for inventory/health;
 bench-result storage and Architecture Decision Records.

Only after the physical/control route is measured should the real adapter be implemented.

11.2 Repository layout

```
/device-farm
/docs
/adr
/bench-results
/runbooks
/apps
/control-api
/operator-web
/node-agent
/packages
/domain
/db
/telemetry
/adapter-sdk
/adapters
/fake
/niaozun      # only after bench/API review
/wda          # optional policy branch
/android-adb
/infra
/docker
/monitoring
/network-reference
/tests
/integration
/hardware-contract
```

11.3 Domain invariants

Claude must encode these rules:

a device belongs to exactly one active pod/node at a time;

- a task targets a device UUID, not a transient USB index;
- a client/platform asset cannot be used without a current authorization record;
- secrets are referenced, never embedded in task payloads/logs;
- vendor adapter calls are auditable;
- destructive actions require stronger authorization and confirmation;
- task retries are bounded and idempotency-aware;
- a quarantined device receives no production tasks;
- operator identity is recorded on every manual action;
- content assets carry client/creator rights/approval state.

11.4 Minimum data model

Client
 Authorization
 PlatformAsset
 Operator
 RoleGrant
 Pod
 HostNode
 Device
 DeviceObservation
 NetworkAssignment
 ContentAsset
 ContentApproval
 Task
 TaskAttempt
 VendorAdapterInstance
 Incident
 AuditEvent
 SecretReference
 BenchExperiment
 ArchitectureDecision

Do not add "ban replacement," "stealth score," or similar evasion-oriented entities. Restrictions/incidents can be logged neutrally for support and reliability.

11.5 Adapter contract

Every device-control backend should implement a small capability-based interface, for example:

```

list_devices()
get_health(device)
capture_screen(device)
input_capabilities(device)
media_capabilities(device)
execute_authorized_action(device, action)
reconnect(device)
collect_diagnostics(device)
  
```

Actions are typed objects, not raw arbitrary scripts from the web UI. This reduces the blast radius of a compromised operator account and makes adapter replacement possible.

11.6 Task state machine

CREATED

-> VALIDATED

-> QUEUED

-> DISPATCHED

-> RUNNING

-> SUCCEEDED

\\-> FAILED_RETRYABLE -> QUEUED (bounded)

\\-> FAILED_FINAL

\\-> HUMAN_REVIEW

Every transition has a timestamp, actor/process, reason and correlation ID.

11.7 First implementation backlog

Create repo, formatter/linter/test pipeline.

Add docs/adr and ADR template.

Define domain IDs and enums.

Create PostgreSQL schema/migrations.

Implement device/pod/host inventory API.

Implement node-agent registration/heartbeat.

Implement fake adapter with deterministic failure injection.

Implement task queue/state machine.

Implement audit-event append path.

Implement observation storage/retention.

Build operator inventory/health page.

Add screenshot object-storage abstraction.

Add RBAC/individual operator accounts.

Add SecretReference interface to chosen vault.

Add bench-experiment/result objects.

Add alerts for lost heartbeat/device enumeration.

Add pod capacity metrics.

Add media-asset/approval model.

Add client/authorization/platform-asset model.

Add device quarantine workflow.

Implement hardware-contract test suite against fake adapter.

After TEST-BOARD-001, implement Niaozun adapter if accepted.

After TEST-MAC-CAPTURE-001, implement capture adapter if public/stable path exists.

Add Android ADB adapter only for owned managed/test devices as needed.

Add remote-operator escalation/live-session link.

Add incident object/runbook links.

Add backup/export of inventory/ADRs/bench data.

Add 5-device pilot deployment manifest.

Run acceptance tests and write results into docs/bench-results.

Create ADR: "Pod size and control method for Pilot v1."

11.8 Definition of done for Pilot v1

five devices continuously represented in inventory;
 stable device identity across host-agent restart;
 per-device heartbeat/health visible;
 task execution through fake + accepted real adapter;
 screenshot/diagnostic capture where supported;
 no plaintext production credentials in database/logs;
 authorized content object can be staged/tested on owned devices;
 individual operator authentication and audit;
 one device can fail/quarantine without blocking others;
 remote host restart/recovery tested;
 power/thermal soak recorded;
 all [OPEN] items that affect Pilot v1 either resolved or explicitly deferred in ADRs.

11.9 "Do not infer" list for Claude

Claude must stop and create a test/ADR rather than assume:
 that Mac USB screen capture scales because QuickTime shows one phone;
 that a vendor board's file transfer writes Photos assets;
 that a 20-port hub supports 20 simultaneous full workloads;
 that resetting a phone clears backend anti-abuse state;
 that pointer input is indistinguishable from touch;
 that a platform actually consumes an available OS signal;
 that a proxy/VPN choice changes recommendation behavior;
 that a specific warmup duration is a documented platform rule;
 that a browser profile isolates platform ownership/billing relationships;
 that changing a password revokes delegated access;
 that an operator's revenue/yield number is independently verified.

11.10 Feedback loop

Every bench result should update three things:

the experiment result file;
 the relevant ADR;
 the handbook's verification ledger at the next revision.

The goal is a living engineering manual whose uncertainty decreases as the physical system is measured.

11.A - Prior Claude Handoff Deep Reference (updated evidence takes precedence)

4. Blocking experiments - run these before major coding or procurement

These tests are ordered by decision impact. Each test should create a markdown report under `/docs/tests/` with hardware IDs, OS/software versions, exact procedure, timestamps, raw evidence references, result, confidence and follow-up. Where a test uses a custom app, keep it limited to owned test software and neutral endpoints.

4.1 TEST-IOS-INPUT-001 - identify the actual iMouse touch type

Field

Specification

Question

Does the exact iMouse board/software path produce direct or indirectPointer touch events in an owned UIKit test app?

Devices

At least one iOS 16-era device if already owned and one iOS 17+ / current-supported device. Do not procure obsolete hardware solely for this test.

Inputs

Finger tap; standard pointing device; iMouse board click.

Test app

Minimal UIKit app logging UITouch.type, event timestamp, coordinates and app build SDK; include visible counter for evidence.

Variants

Build once with UIApplicationSupportsIndirectInputEvents omitted, once YES, once NO.

Evidence

Device screen recording, app log, iOS version, app SDK version, iMouse version, board ID.

Pass condition

Actual event type for each combination is documented and repeatable.

Decision effect

Determines whether iMouse is acceptable as a human remote-control method and what observability caveat must be documented. It is not a basis for evasion.

4.2 TEST-NET-EGRESS-001 - Wi-Fi control plane versus cellular data plane

Field

Specification

Question

Can the phone maintain local mirroring/control on the control WLAN while normal Internet access leaves over cellular, and is that stable?

Topology

Dedicated AP/SSID and controller on the same private subnet. First run with normal Internet gateway; second run with WLAN Internet egress intentionally unavailable while local subnet remains reachable.

Measurement

Neutral HTTPS endpoint owned by XYZ logs source address; router records WAN counters; device records cellular-data delta; iMouse mirror stability is logged.

Scope

Test Safari/owned diagnostic app. Do not infer a third-party app route without independent observation.

Evidence

Endpoint log, router counters, cellular counter screenshots, mirror uptime, route conditions.

Pass condition

A stable, documented combination is found or the requirement is rejected as unreliable.

Decision effect

Selects normal Wi-Fi, cellular, or managed VPN architecture. Do not treat the result as an anti-detection feature.

4.3 TEST-IOS-CONTROL-001 - iMouse stability and security baseline

- Install on a dedicated Windows test host, not a personal workstation.
- Record installer hash, version, services, listening ports, startup behavior and update mechanism.
- Keep endpoint protection enabled. If a narrow exclusion is genuinely required, document exactly which folder/process and why; do not uninstall security software as a default.
- Bind two phones and run mirror/control continuously. Record reconnects, dropped frames, CPU/GPU/RAM/NIC, USB board disconnects and service restarts.
- Confirm whether API endpoints have authentication/TLS. If not, enforce host firewall + management-VLAN ACL and bind/allow-list only required addresses.
- Exercise only neutral actions in the test: screen mirror, screenshot, clipboard test, safe settings navigation, media import to owned test assets.

4.4 TEST-MEDIA-IOS-001 - content delivery to Photos

The vendor API states that photos/videos can be sent to the phone album. Validate the exact mechanism, format constraints, import latency, duplicate behavior, metadata preservation and failure recovery on two current devices. The goal is a reliable cloud -> queue -> local node -> device Photos import workflow. Do not tie this test to an automated social-post action.

4.5 TEST-CAPACITY-001 - devices per Windows node

There is no credible universal “devices per host” constant. The capacity boundary depends on mirroring resolution/fps, codec, CPU/GPU decode, NIC, AirPlay implementation, iMouse process behavior, USB controller topology and operator UI. Benchmark 1, 5, 10 and 20 devices on the actual host. Only test 40 on one host if 20 is comfortably below resource and failure thresholds; otherwise scale by pods.

Metric

Collect at each concurrency level

Initial engineering gate

CPU/GPU

mean, p95, peak; per-process if available

p95 < 70% recommended starting gate

RAM

working set, commit, paging

no sustained paging; >25% headroom

NIC

RX/TX Mbps, drops, errors

<60% sustained link utilization; no drops

Mirror quality

fps, reconnect count, time-to-first-frame

no unexplained reconnect storm

Control latency

click-to-visible-result in owned test UI

p95 target defined from operator usability test

Node recovery

restart iMouse service / agent / host

devices recover without database corruption

USB

board enumeration, disconnects by controller/hub

no silent device loss over soak test

The utilization percentages above are design starting points, not vendor guarantees. Replace them with local measurements after the first soak test.

4.6 TEST-THERMAL-001 - 20-slot thermal and charging characterization

Use the actual chassis. Run empty, 5-device, 10-device and full-load stages. Log ambient intake/exhaust, representative device temperature indications, charging state, fan state and power draw. Apple specifies 0-35 C ambient operation and can slow or pause charging when temperature is high; the rack should target substantially below the upper limit to preserve battery life and headroom.

Sources: [P18-P19]

5. Recommended reference architecture - durable iOS pod

Figure A2 - Recommended control-plane architecture. The vendor controller remains local; XYZ owns the authenticated outer control plane.

5.1 Pod as the scaling unit

Treat one physical pod as the smallest independently recoverable unit. A pod contains 5-20 phones, one dedicated Windows node, one control-board/chassis set, one control WLAN segment, labeled power distribution and a telemetry identity. A failure in one pod should not take down another pod. Scaling from 20 to 40 devices means adding a second pod unless the capacity test proves that a single node remains comfortably within thresholds and the blast radius is acceptable.

5.2 Component responsibilities

Component

Responsibility

Must not do

Operations dashboard

Inventory, health, queues, operator attention, audit search, approval views

Store platform passwords in browser local storage; directly expose vendor API

Control API

AuthN/AuthZ, business rules, task creation, state transitions
 Issue unrestricted arbitrary mouse macros to third-party apps
 PostgreSQL
 Authoritative inventory, assignments, tasks, audit metadata
 Store raw secrets in plaintext
 Redis
 Short-lived task leases, heartbeats, event fan-out
 Become source of truth
 Object storage
 Approved media, screenshots/evidence, logs/artifacts
 Serve unapproved content to devices
 Windows node agent
 Local adapter to iMouse, media staging, health collection
 Accept unauthenticated commands from the Internet
 iMouse local service
 Mirror/control hardware and vendor-supported device operations
 Be Internet-exposed or trusted as the global orchestration layer
 MDM / ABM
 Supervision, enrollment, restrictions, managed apps, inventory/policy
 Replace the app/platform's own authorization rules
 Phone
 Human-operated publishing endpoint / platform client where legitimately required
 Be treated as disposable identity material

5.3 Task taxonomy

The control plane should implement an allow-list of task types. This prevents a general-purpose device-control API from quietly becoming a platform automation engine.

Task type

Automation level

Evidence requirement

health_check

Automatic

heartbeat + state snapshot

capture_screenshot

Automatic for diagnostics

reason + retention policy

import_media

Automatic after asset approval

asset hash + approval ID + device

sync_inventory

Automatic

MDM/vendor state

restart_local_control_service

Automatic with retry budget

incident/task ID

request_human_action

Human

operator ID + start/end + outcome

open_owned_test_app

Automatic in lab only

test ID

third_party_social_ui_action

Human by default

authorization + operator audit; no autonomous macro library

account_creation / bulk friend-add / fake engagement / enforcement bypass

Blocked

no implementation

5.4 Device lifecycle

Figure A3 - Device lifecycle state machine.

Do not overload “banned,” “broken,” “offline,” and “retired.” They are different states with different responses. A platform enforcement event belongs to the account/device incident record; it must not automatically trigger account replacement. A hardware fault belongs to maintenance. A security concern moves the device/node to quarantine. Retirement is a deliberate asset lifecycle action.

10. Software control plane - concrete implementation specification

Figure A5 - Safe task execution flow with policy gate and human path.

10.1 Reference stack

Layer

Recommended technology

Reason

Backend API

Python 3.12 + FastAPI + Pydantic

Matches iMouse Python ecosystem, strong typed APIs, rapid integration.

ORM/migrations

SQLAlchemy 2 + Alembic

Mature Postgres integration and explicit schema migration.

Database

PostgreSQL 16+

Durable source of truth, JSONB where needed, strong constraints.

Ephemeral queue/cache

Redis 7+

Task leases, heartbeats, short-lived coordination; not source of truth.

Frontend

Next.js + TypeScript

Operations dashboard, SSO integration, typed client.

Object storage

S3-compatible (cloud S3 or MinIO for local pilot)

Media/evidence storage with lifecycle policies.

Node agent

Python Windows service

Local vendor adapter and telemetry collector.

Metrics

Prometheus/OpenTelemetry-compatible metrics

Standard fleet/node observability.

Logs

Structured JSON -> centralized collector

Searchable incident evidence.

Secrets

Cloud secret manager or Vault-class service

No secrets in DB/source/config files.

Packaging

Docker for backend services; native Windows service for hardware node

USB/AirPlay vendor controller should remain native on node.

10.2 Repository layout

phone-farm-control/

README.md

SECURITY.md

docs/

architecture/

decisions/ # ADR-0001 ...

tests/ # TEST-IOS-INPUT-001 ...

runbooks/

source-ledger.md

unknowns.md

backend/

app/

api/

domain/

services/

adapters/

models/

security/

migrations/

tests/

dashboard/

node-agent/

adapters/

imouse/

android/

telemetry/

service/

tests/

infra/

docker/

monitoring/

network/

schemas/

events/

api/

scripts/
dev/
ops/

10.3 Architectural boundaries

- `domain/` never imports the iMouse Python package. It speaks in semantic operations such as `ImportMedia`, `CaptureDiagnosticScreenshot`, `GetDeviceHealth` and `RequestHumanAttention`.
- `node-agent/adapters/imouse` is the only module that knows iMouse endpoint names, board IDs and vendor response formats.
- `node-agent/adapters/android` uses ADB/scrcpy only for authorized engineering/device operations.
- Third-party platform credentials are referenced via secret IDs and authorization records, not embedded in tasks.
- Every task is idempotent or explicitly non-retryable; the system never repeats a social/publishing action merely because the network response was lost.
- Every state mutation appends an audit event; mutable “current state” is derived from authoritative tables plus events.

10.4 Device data model

Figure A6 - Minimal entity relationships.

```

device
id UUID PK
asset_tag TEXT UNIQUE
platform ENUM(iOS, Android)
model TEXT
os_version TEXT
serial_ref TEXT NULL # internal inventory ref, not exposed broadly
rack_id UUID
slot_no INT
lifecycle_state ENUM
mdm_ref TEXT NULL
acquired_at TIMESTAMPTZ
retired_at TIMESTAMPTZ NULL

control_endpoint
device_id UUID FK
node_id UUID FK
adapter ENUM(imouse, android)
vendor_device_ref TEXT
board_ref TEXT NULL
last_seen_at TIMESTAMPTZ
health_state ENUM

account_assignment
id UUID PK
device_id UUID FK
platform TEXT
account_external_ref TEXT
creator_id UUID
authorization_ref TEXT
status ENUM(active, paused, review, ended)
secret_ref TEXT NULL

```

```

asset
id UUID PK
creator_id UUID
sha256 TEXT UNIQUE
object_key TEXT
media_type TEXT
rights_status ENUM
approval_state ENUM
safety_classification TEXT

task
id UUID PK
type TEXT
device_id UUID NULL
asset_id UUID NULL
state ENUM(queued, leased, running, human_wait, succeeded, failed, cancelled)
idempotency_key TEXT UNIQUE
requested_by UUID
policy_decision JSONB
lease_expires_at TIMESTAMPTZ NULL

audit_event
id BIGSERIAL PK
occurred_at TIMESTAMPTZ
actor_type TEXT
actor_id TEXT
event_type TEXT
entity_type TEXT
entity_id TEXT
result TEXT
evidence_ref TEXT NULL
metadata JSONB

```

10.5 Task API examples

```

POST /v1/tasks
{
  "type": "import_media",
  "device_id": "...",
  "asset_id": "...",
  "idempotency_key": "import:<device>:<asset>:v1"
}

POST /v1/tasks
{
  "type": "request_human_action",
  "device_id": "...",
  "reason": "Publish approved item using authorized account",
  "approval_ref": "APR-2026-..."
}

GET /v1/devices?rack_id=...&health=degraded
GET /v1/tasks/{id}
POST /v1/tasks/{id}/cancel
GET /v1/audit?device_id=...&from=...&to=...

```

10.6 Policy engine

Before a task can be leased to a node, evaluate a small explicit policy. This is not a generic AI judgment. It is deterministic code backed by authorization and asset state.

```
allow(task) only if:
device.lifecycle_state in {AVAILABLE, ASSIGNED, ACTIVE}
AND node.health == HEALTHY
AND task.type in ALLOWED_TASK_TYPES
AND (asset is null OR asset.approval_state == APPROVED)
AND (account is null OR account.authorization_ref is valid)
AND no quarantine/security hold exists
AND any human-required task has assigned operator

BLOCKED_TASK_TYPES include:
bulk_account_creation
bulk_follow_or_friend_add
fake_engagement
fingerprint_spoofing
enforcement_bypass
automatic_replacement_after_platform_ban
```

11. Node agent and vendor-adapter design

11.1 Node-agent responsibilities

- Authenticate to backend with a unique node certificate and rotate credentials.
- Register hardware/software version, node health, NICs and adapter capabilities.
- Lease only tasks assigned to its pod and renew leases while executing.
- Validate task schema and local policy before calling the vendor adapter.
- Stage approved media in an ephemeral local directory; verify SHA-256 before import.
- Collect adapter/device health and send structured heartbeats.
- Upload diagnostic evidence through signed object-storage URLs.
- On restart, reconcile leased tasks rather than blindly repeating them.
- Never persist platform passwords in logs, screenshots directory names or crash dumps.

11.2 Adapter interface

```
class DeviceAdapter(Protocol):
def list_devices(self) -> list[DeviceStatus]: ...
def get_health(self, device_ref: str) -> DeviceHealth: ...
def capture_diagnostic_screenshot(self, device_ref: str) -> bytes: ...
def import_media(self, device_ref: str, local_path: str) -> ImportResult: ...
def restart_control_channel(self, device_ref: str) -> RestartResult: ...

# Deliberately not part of the generic production interface:
# arbitrary_click_macro()
# bulk_account_create()
# auto_follow_users()
# rotate_identity_after_ban()
```

11.3 Heartbeat contract

```
{
"node_id": "pod-01-node",
```

```

"timestamp": "2026-08-26T18:10:00Z",
"agent_version": "0.1.0",
"adapter": {"name": "imouse", "version": "<measured>"},
"host": {"cpu_pct": 32.4, "ram_pct": 41.0, "rx_mbps": 18.2, "tx_mbps": 4.1},
"devices": [
{"device_id": "ios-p1-s01", "connected": true, "mirror": true, "last_frame_ms": 85},
{"device_id": "ios-p1-s02", "connected": true, "mirror": true, "last_frame_ms": 91}
]
}

```

11.4 Retry semantics

Operation

Retry policy

Read health/list devices

Safe exponential retry with bounded timeout.

Capture diagnostic screenshot

Safe bounded retry; dedupe evidence by task ID.

Import media

Retry only if adapter provides deterministic success check; verify Photos presence/hash surrogate where possible.

Restart local service

One automatic attempt, then human/incident escalation.

Human third-party UI action

Never auto-retry; operator must confirm outcome.

Unknown vendor error

Fail closed and preserve raw vendor error in restricted diagnostic field.

11.5 Local security boundary

The node agent should connect outward to the backend; the backend should not open arbitrary inbound administrative ports on the Windows node. If inbound management is required, use a corporate VPN/zero-trust access layer with device identity and MFA. Keep iMouse itself bound to localhost or the control VLAN as narrowly as the product permits. Use the vendor `allow\_ip\_list` feature if validated, but do not treat it as the sole security control.

15. Build plan and stage gates

Figure A7 - Scale only after evidence gates pass.

Gate 0 - inventory and architecture intake

1. Meet with Dim and capture the exact tested control software, version, installer source, screenshots/video, hardware board, cable topology and observed limits.
2. Inventory all phones: model, OS, battery condition, lock/ownership state, SIM capability, connector, MDM eligibility.
3. Inventory the 20-slot enclosure: vendor/model, PSU label, fans, board types, port topology, required host connection.
4. Create `docs/unknowns.md` with every unresolved claim from the internal brief.
5. Create ADR-0001 “Durable pod first” and ADR-0002 “No autonomous third-party social UI automation.”

Gate 1 - two-device evidence bench

6. Run TEST-IOS-INPUT-001.
7. Run TEST-NET-EGRESS-001.
8. Run TEST-IOS-CONTROL-001.
9. Run TEST-MEDIA-IOS-001.
10. Capture Windows/iMouse ports, service dependencies and security exceptions.
11. Decide iMouse vs Dim-tested alternative with ADR-0003.

Gate 2 - five-device pilot pod

12. Build the backend skeleton and PostgreSQL schema.
13. Implement node registration/heartbeats and `imouse` adapter for health, screenshot and approved media import.
14. Build device inventory UI and physical slot mapping.
15. Implement creator/account authorization references and asset approvals.
16. Implement human session lock + outcome reporting.
17. Run 5-device soak and collect power/thermal/network/controller metrics.
18. Complete security review before storing real creator media.

Gate 3 - ten-device stable pod

19. Expand only by duplicating the known working wiring/slot pattern.
20. Run capacity tests and node restart/recovery drills.
21. Add backup/restore, alerts, operator roles and offboarding.
22. Measure actual operator time per approved publish task and media-import latency.
23. Review platform rules again and freeze the pilot operating SOP.

Gate 4 - twenty-device production pod

24. Complete full chassis thermal/power soak.
25. Document spare ratio and replacement procedure.
26. Enforce SSO/MFA and production secret management.
27. Add business attribution dashboards only from authorized tracking data.
28. Complete go/no-go review for a second pod. Do not jump to 100+ because one 20-device pod works.

Gate 5 - multi-pod scale

Scale horizontally: pod-01, pod-02, pod-03. Each pod has a unique node certificate, control subnet/SSID identity, rack IDs, monitoring labels and failure domain. Central services schedule only to healthy pods. Keep a staging pod for controller/vendor updates before production rollout.

16. Acceptance criteria by subsystem

16.1 Physical pod

- Every slot, phone, control board, cable group and power output is labeled and represented in inventory.
- Full-load thermal soak completes without thermal charging hold, device shutdown, abnormal battery behavior or unplanned controller disconnect attributable to heat.
- Power draw is measured at idle, active mirroring and simultaneous charging; PSU and upstream circuit have documented headroom.
- One phone can be removed/replaced without disturbing neighboring devices.
- Emergency shutdown and battery incident procedure is posted and tested.

16.2 Windows/iMouse control node

- Node can cold boot and restore all healthy device mirrors without manual undocumented steps.
- Vendor service ports are unreachable from guest/employee networks and the public Internet.
- Endpoint security remains enabled with documented narrow exceptions only if needed.
- Agent can detect vendor service failure, attempt one controlled restart, then escalate.
- No vendor API credential or node certificate is committed to source control.
- A second engineer/operator can reproduce installation from the runbook.

16.3 Backend/control plane

- PostgreSQL is the source of truth; Redis loss does not lose inventory or completed audit history.
- Duplicate task submissions with same idempotency key do not create duplicate effects.
- Every task outcome is attributable to actor, device, asset and authorization context.
- Blocked task types cannot be created by normal API clients.
- RBAC prevents a staff member assigned to creator A from browsing creator B media unless explicitly authorized.
- Backup restoration is tested, not merely configured.

16.4 Operations

- An operator can identify which device needs attention in under one minute from the dashboard.
- No one needs a spreadsheet to determine current phone/account/slot ownership.
- Approved content reaches the intended phone with hash/asset traceability.
- Platform warnings are captured as incidents and stop further automated internal tasks for the affected assignment until review.
- Creator exit can revoke account assignment and media access without dismantling the whole fleet.

18. Risk register

ID

Risk

Likelihood

Impact

Mitigation / decision

R-01

Control vendor requires unsafe endpoint-security exclusions

M

H

Dedicated node, narrow exception, isolation, staging, hashes; reject if containment inadequate.

R-02

iMouse input is distinguishable in actual app path

M

M/H

Measure in owned test app; do not rely on invisibility; human-only use or alternate control.

R-03

Control WLAN captures Internet egress unexpectedly

M

M

Run egress test; choose documented routing; never assume SIM.

R-04

Platform app raises minimum OS and strands old fleet

H over lifecycle

H

Buy current-supported models; update ring; app compatibility watch.

R-05

Battery heat/swelling in dense rack

M

H

Thermal design, inspection, managed charging, power telemetry, remove damaged cells.

R-06

Vendor controller instability at 20+

M

M/H

Capacity test; pod isolation; horizontal scale.

R-07

Operator claim drives over-purchase before replication

M

H

Stage gates and n=5/n=10 evidence; no 100+ procurement from testimony.

R-08

Reset assumed to erase platform device history

M

H

Document DeviceCheck/device recall reality; treat enforcement as incident, not reset workflow.

R-09

Media/credential cross-creator leak

M

H

RBAC, separate prefixes, secret manager, audits, creator exit procedure.

R-10

AI coding agent adds prohibited generic automation

M

H

Blocked task list, code review, policy tests, SECURITY.md.

R-11

Platform terms change

H

M/H

Quarterly source refresh + pre-launch review.

R-12

Single Windows node becomes fleet-wide blast radius

M

H

20-device pods, separate certificates/segments, staged updates.

19. Architecture Decision Records Claude must create

ADR

Decision

Required evidence before closing

ADR-0001

Durable pod first vs disposable fleet

Internal readiness + platform policy + operator evidence classification.

ADR-0002

Third-party UI automation boundary

Platform rules and project safety boundary.

ADR-0003

Dim-tested controller vs iMouse

Hands-on comparison, security, API, stability, host OS, cost.

ADR-0004

iOS device acquisition floor

Current Apple OS support + current app compatibility + price.

ADR-0005

20 devices per pod

Capacity/thermal/power measurements.

ADR-0006

Wi-Fi vs cellular egress design

TEST-NET-EGRESS-001.

ADR-0007

MDM provider / enrollment method

ABM eligibility, supervised feature needs, API/export/cost.

ADR-0008

Backend stack

Team skills, Windows-node integration, deployment target.

ADR-0009

Object storage and media retention

Creator authorization/privacy/backup needs.

ADR-0010

Android pilot yes/no

Actual platform need vs engineering convenience.

ADR-0011

Proxy/VPN use if any

Documented security/business requirement; provider terms.

ADR-0012

Production update strategy

Staging pod, OS app compatibility, vendor controller updates.

ADR template

ADR-XXXX - <title>

Status: Proposed | Accepted | Rejected | Superseded

Date: YYYY-MM-DD

Owners: ...

Context

What decision is required and why now?

Evidence

Internal observations/test IDs

Primary-source links

Vendor facts

Assumptions explicitly labeled

Options

A ...

B ...

C ...

Decision

Chosen option and exact scope.

Consequences

Benefits, costs, risks, operational load.

Rollback / exit

How to reverse without losing fleet state.

Follow-up

Tests, procurement, code changes, review date.

20. Claude execution backlog - first 30 work items

ID

Work item

Acceptance evidence

PF-001

Interview Dim and capture tested controller evidence
docs/source-ledger.md updated; exact product/version/hardware known.

PF-002

Inventory existing 20-device enclosure
Photos, PSU/fan/port specs, slot map.

PF-003

Inventory existing phones
CSV/database import with model/OS/battery/ownership state.

PF-004

Create unknowns ledger
Every §8/§11 internal-brief unknown mapped to owner/test.

PF-005

Create ADR-0001/0002
Architecture/safety baseline accepted.

PF-006

Create minimal UIKit input test app
Logs touch type with build/version evidence.

PF-007

Run TEST-IOS-INPUT-001
Report and artifacts complete.

PF-008

Build isolated two-device control WLAN
No public exposure; diagrams and ACLs documented.

PF-009

Run TEST-NET-EGRESS-001
Route behavior documented.

PF-010

Install chosen iOS controller on dedicated Windows host

Hash/version/services/ports captured.

PF-011

Run controller security review

Firewall/AV exception decision documented.

PF-012

Run 2-device mirror/control soak

Reconnect/resource metrics collected.

PF-013

Run media import test

Format/latency/failure behavior documented.

PF-014

Create backend repository and CI

Lint/test/migrations pipeline green.

PF-015

Implement PostgreSQL schema

Migration + constraints + test fixtures.

PF-016

Implement auth/RBAC skeleton

Roles and creator scoping tests.

PF-017

Implement node registration/mTLS plan

Unique node identity and revocation path.

PF-018

Implement heartbeat endpoint

Dashboard can show stale/healthy nodes.

PF-019

Implement iMouse adapter health/list

No arbitrary macro endpoint.

PF-020

Implement approved media import task

Hash checked, audited, idempotent.

PF-021

Implement diagnostic screenshot task

Retention and reason captured.

PF-022

Implement human session lock

One operator/device at a time, outcome logged.

PF-023

Build device/rack dashboard

Physical slot and assignment visible.

PF-024

Build asset approval model

Unapproved media cannot be queued.

PF-025

Build account authorization model

Assignment requires authorization_ref.

PF-026

Assemble five-device pod

Known BOM/wiring labels.

PF-027

Run five-device power/thermal/network soak

Measured capacity sheet produced.

PF-028

Run failure drills

Node agent restart, controller restart, DB/Redis outage behavior.

PF-029

Review current platform rules

Source ledger dated before pilot operations.

PF-030

Gate-2 review

Written go/no-go for 10-device expansion.

21. Claude startup prompt embedded in the handoff

You are the technical lead for XYZ MGMT's physical device-fleet project.

Treat Part A of this PDF as the current architecture authority and Part B as the deep engineering reference.

Before writing production code:

Read the READ THIS FIRST section and the verification ledger.

Create an Unknowns Ledger and ADRs.

Obtain the missing hands-on facts from Dim; do not guess the name/version of his tested control product or what hardware is already owned.

Close the blocking input-type, egress, controller-security and media-import tests on a two-device bench.

Build the durable 5-device pilot before any 20+ scale decision.

Do not implement or optimize: bulk account creation, bulk friend/follow/add behavior, fake engagement, scraped-user acquisition, fingerprint spoofing, location falsification, automatic ban replacement, CAPTCHA/rate-limit bypass, or a generic third-party social-UI macro engine.

Use a typed control plane: FastAPI + PostgreSQL + Redis + a Windows node agent + vendor adapters. Keep the vendor iOS controller local and isolated. Use MDM for governance and the local controller for human remote operation/approved media handling.

For every task you propose, include: evidence/source, dependencies, exact output artifact, acceptance criteria, rollback, and which ADR/test it closes.

If evidence conflicts, separate observation from interpretation and flag the conflict rather than silently reconciling it.

24. Final handoff checklist

☐ What hardware and software does XYZ already own?

☐ Which controller is selected, and why is it better than the alternative actually tested by Dim?

- ☐ What are the controller's listening ports, trust boundaries and endpoint-security requirements?
- ☐ What event type does the controller produce in the owned iOS input test app?
- ☐ What interface carries Internet traffic under the selected control topology?
- ☐ What is the measured devices-per-node capacity at target mirror quality?
- ☐ What is the full-load power draw and thermal profile of one pod?
- ☐ What iPhone model floor is being purchased and what current OS/app-support evidence supports it?
- ☐ How are devices supervised/managed and wiped/retired?
- ☐ How are approved media assets imported without losing attribution/audit information?
- ☐ How does a human operator take exclusive control of one device and report the outcome?
- ☐ Which actions are explicitly blocked from automation?
- ☐ How are credentials, media and creator boundaries protected?
- ☐ How is a platform warning/enforcement event handled without automated circumvention?
- ☐ What are the 12-month pod TCO and measured operator minutes per useful outcome?
- ☐ What evidence is required before 5 -> 10, 10 -> 20, and 20 -> multi-pod expansion?

Part B follows

APPENDICES - Reference Configurations, Checklists, Formulas and Glossary

APPENDIX A - Example IP/VLAN plan

Example only. Choose ranges that do not conflict with the organization's existing network or VPNs.

VLAN

Purpose

Example subnet

Allowed inbound

Primary egress

10

Management

10.20.10.0/24

Admin/VPN only

Direct corporate

20

Phone fleet A

10.20.20.0/24

Mgmt services only

Policy-defined

21

Phone fleet B

10.20.21.0/24

Mgmt services only

Policy-defined

30

Data/storage

10.20.30.0/24

API/worker only

Restricted

40

Quarantine

10.20.40.0/24

Mgmt only

Updates/support only

50

Guest

10.20.50.0/24

None

Direct isolated

Do not expose MDM connectors, ADB services, broker ports, database or storage administration directly to the public internet. Administrative access should traverse a secured management path.

APPENDIX
B - Example
firewall
policy

Source	Destination	Service	Action	Reason
Mgmt VLAN	Fleet API	HTTPS	Allow	Admin/dashboard
Device VLAN	DNS resolvers	DNS/DoH/DoT as designed	Allow	Name resolution
Device VLAN	NTP/time	NTP/HTTPS time	Allow	Clock sync
Internet	HTTPS + required platform traffic			

Allow
 Workload
 Device VLAN
 Mgmt VLAN
 Any
 Deny
 Prevent lateral access
 Device VLAN
 Device VLAN
 Any
 Deny by default
 Limit east-west
 Quarantine
 Internet
 Updates/support subset
 Allow limited
 Recovery
 Internet
 Database/ADB/broker admin
 Any
 Deny
 No direct exposure

The exact service list depends on Apple/Google management requirements, VPN/MDM vendor endpoints and platform applications. Use vendor-published network requirements and validate with staged tests.

APPENDIX C - Task JSON and API examples

These examples describe the internal coordinator. They do not contain commands for automating third-party social user interfaces.

Recommended API safeguards

- OAuth/OIDC for employees; service credentials for connectors.
- Role and creator/account scope on every request.
- Idempotency keys for create/publish requests.
- Request ID and actor in every audit event.
- Schema validation and bounded text lengths.
- No plaintext platform passwords in request bodies.
- Rate limits for internal clients.

APPENDIX D - Example SQL data model

Store sensitive serial/IMEI/recovery metadata with access controls or separate encrypted references as appropriate. The schema is illustrative and omits foreign keys/indexes for readability.

APPENDIX E - MQTT topic design

Authorization model

- A device may publish only to its own health/events/result topics.
- A device may subscribe only to its own command topic.
- Rack sensors may publish only to their rack namespace.
- Operators use the API, not direct broker credentials.
- Broker admin interface is management-network only.
- TLS client authentication or strong per-device credentials.

APPENDIX F - Device commissioning checklist

Receiving

- ☐ Asset tag assigned
- ☐ Serial/model recorded
- ☐ Activation/carrier locks checked
- ☐ Battery/charging port inspected
- ☐ Factory reset performed

Enrollment

- ☐ Correct owner/enterprise
- ☐ MDM/Android Enterprise enrollment complete
- ☐ Supervision/fully-managed state verified
- ☐ Baseline policy applied
- ☐ Required apps installed

Network

- ☐ Correct SSID/VLAN
- ☐ DNS works
- ☐ VPN/proxy policy verified if applicable
- ☐ Management heartbeat received
- ☐ Public egress recorded for diagnostics

Security

- ☐ Passcode/lock policy
- ☐ Disk/device encryption active by platform default/management
- ☐ Debugging disabled unless required
- ☐ Credential vault assignment complete
- ☐ Lost/wipe procedure tested on cohort

Operations

- ☐ Rack slot assigned
- ☐ Cable/port mapped
- ☐ Tracking/companion app check-in

[] Test task completed

[] Status set to Ready

APPENDIX G - Weekly operations report template

Section

Required fields

Executive summary

What improved, what deteriorated, major risk/blocker, next decision

Fleet health

Online %, degraded/offline devices, thermal/power incidents, update status

Publishing

Tasks completed/failed, failure causes, approval latency, executor split

Account health

Restrictions/notices, appeals, account availability, policy changes

Funnel

Reach where available, clicks, conversion, revenue attribution confidence

Content

Top/bottom assets, experiments, QC/rejection rate

Security

Access changes, incidents, credential rotations, overdue reviews

Cost

Labor hours, connectivity/software, repair/amortization, estimated contribution

Next week

Priorities, experiments, maintenance, owner and deadline

APPENDIX H - Incident runbooks

Lost or stolen phone

1. Mark device Quarantined in coordinator.
2. Use MDM/Enterprise controls to lock/wipe as appropriate.
3. Revoke internal device token and any account sessions/credentials accessible from the device.
4. Preserve inventory/last-seen/network/task logs.
5. Notify creator/security owner if their account/content was exposed.
6. Replace from spare pool and reassign only after incident review.

Credential compromise

7. Disable/revoke compromised account or delegated session.
8. Rotate password/API token and MFA recovery material as required.
9. Search audit logs for access/use during exposure window.
10. Revoke employee/service sessions that inherited the secret.
11. Restore only after clean admin workstation and access path are verified.

Rack overheating

12. Stop new workload.
13. Inspect fan/airflow and ambient conditions.
14. Isolate abnormally hot/swollen devices; disconnect safely.
15. Reduce charging load or power down affected zone.
16. Do not return devices until temperatures and root cause are normal.

Platform restriction

17. Preserve notice and post/account IDs.
18. Pause similar queued tasks.
19. Classify policy/security/copyright/age-consent cause.
20. Use legitimate remediation/appeal path.
21. Update campaign rules; do not auto-create replacement identities.

Database outage

22. Pause writes/publishing that cannot be reconciled.
23. Verify storage/host health.
24. Fail over or restore according to runbook.
25. Reconcile queue vs database after recovery.
26. Document RPO/RTO achieved and any lost events.

APPENDIX I - Capacity formulas

Electrical planning

Storage planning

Task throughput

Wi-Fi

Do not use a single “devices per AP” constant. Capacity depends on airtime, channel width, modulation/rate, interference, application traffic, AP radio design and environment. Measure channel utilization, retries and application latency under representative concurrent load.

APPENDIX J - Glossary

Term

Meaning

ADB

Android Debug Bridge; Google development/diagnostic bridge for authorized Android devices.

ACL

Access Control List; rule set permitting/denying network flows.

AP

Wireless access point.

APNs

Apple Push Notification service; used by MDM to signal devices that management work is available.

CGNAT

Carrier-Grade Network Address Translation; NAT performed in a carrier network at subscriber scale.

Control plane

Systems that manage policy, tasks, identity, inventory and administration.

Data plane

Actual media/application/network traffic produced by workload execution.

Dedicated device

Android Enterprise fully managed device locked to one or a small set of work purposes/apps.

DoH

DNS over HTTPS.

DoT

DNS over TLS.

Egress

Outbound path/address through which traffic leaves a network.

Idempotency

Property that allows the same logical request to be retried without creating duplicate effects.

MDM

Mobile Device Management.

MQTT

Lightweight publish/subscribe messaging protocol.

NAT

Network Address Translation.

PDU

Power Distribution Unit.

Proxy

Intermediary that makes/relays application connections on behalf of a client.

RBAC

Role-Based Access Control.

RPO

Recovery Point Objective; maximum tolerable data loss window.

RTO

Recovery Time Objective; target service restoration time.

Supervision

Apple organization-owned device management state providing additional controls.

TUN

Virtual network interface commonly used by VPN software.

VLAN

Virtual LAN; logical Layer-2 network segmentation.

VPN

Virtual Private Network; encrypted/logical tunnel to a network gateway.

APPENDIX K - Research references

Research cut: 26 August 2026. First-party sources are preferred. Secondary/archived sources are labeled. Platform policies can change; verify again before implementation.

ID

Source

URL / note

Quality

S1

Google Android Management API - Enroll and provision a device

<https://developers.google.com/android/management/provision-device>

First-party

S2

Google Android Management API - Policies reference

<https://developers.google.com/android/management/reference/rest/v1/enterprises.policies>

First-party

S3

Android Developers - Android Debug Bridge (adb)

<https://developer.android.com/tools/adb>

First-party

S4

Android Developers - VPN / VpnService

<https://developer.android.com/develop/connectivity/vpn>

First-party

S5

Apple Platform Deployment

<https://support.apple.com/guide/deployment/welcome/web>

First-party

S6

Apple Developer - Device Management

<https://developer.apple.com/documentation/devicemanagement/>

First-party

S7

Apple Developer - MDM commands and queries

<https://developer.apple.com/documentation/devicemanagement/commands-and-queries>

First-party

S8

Apple Platform Deployment - supervision

<https://support.apple.com/guide/deployment/dep1d89f0bfff/web>

First-party

S9

AWS Device Farm Developer Guide

<https://docs.aws.amazon.com/devicefarm/latest/developerguide/welcome.html>

First-party; comparison/testing reference

S10

USB-IF - USB Charger / USB Power Delivery

<https://www.usb.org/usb-charger-pd>

First-party standards body

S11

Apple - Keeping iPhone/iPad within acceptable operating temperatures

<https://support.apple.com/en-gb/118431>

First-party

S12

Apple - iPhone battery and charging

<https://support.apple.com/en-gb/105105>

First-party

S13

Cisco - Wireless High Client Density Design Guide

https://www.cisco.com/c/en/us/td/docs/wireless/controller/technotes/8-7/b_wireless_high_client_density_design_guide.html

First-party vendor design guide

S14

Cloudflare - Forward/reverse proxy explanation

<https://www.cloudflare.com/learning/cdn/glossary/reverse-proxy/>

Vendor educational reference

S15

Cloudflare - A primer on proxies

<https://blog.cloudflare.com/a-primer-on-proxies/>

Vendor technical reference

S16

Cloudflare - DNS security

<https://www.cloudflare.com/learning/dns/dns-security/>

Vendor educational reference

S17

Cloudflare - DNS over TLS vs DNS over HTTPS

<https://www.cloudflare.com/learning/dns/dns-over-tls/>

Vendor educational reference

S18

APNIC - Carrier Grade NAT training material

<https://training.apnic.net/wp-content/uploads/sites/2/2016/12/TIP602.pdf>

Regional Internet registry training

S19

WireGuard official site

<https://www.wireguard.com/>

First-party project

S20

MQTT official site

<https://mqtt.org/>

Standards/project reference

S21

Prometheus - Overview

<https://prometheus.io/docs/introduction/overview/>

First-party project

S22

TikTok for Developers - Content Posting API

<https://developers.tiktok.com/doc/content-posting-api-get-started>

First-party

S23

TikTok Community Guidelines - Integrity and Authenticity

<https://www.tiktok.com/community-guidelines/en/integrity-authenticity/>

First-party

S24

Reddit Rules

<https://redditinc.com/policies/reddit-rules>

First-party

S25

Reddit Data API Terms

<https://redditinc.com/policies/data-api-terms>

First-party

S26

Snap - Harmful False or Deceptive Practices

<https://values.snap.com/policy/policy-community-guidelines/harmful-false-deceptive-practices>

First-party

S27

X Help - Authenticity

<https://help.x.com/en/rules-and-policies/authenticity>

First-party

S28

UK Parliament written evidence quoting historical OnlyFans terms

<https://committees.parliament.uk/writtenevidence/40780/default/>

Official public record; historical

S29

Archived OnlyFans Terms snapshot

<https://archive.ph/6mux7>

Archive/secondary; verify current terms directly

S30

2026 secondary analysis of OnlyFans agency terms

<https://whalefinders.com/blog/does-onlyfans-allow-agencies-tos-rules-2026>

Secondary; unverified interpretation

S31

OSHA - Extension Cords and Power Strips Safety at Work (2026)

<https://www.osha.gov/sites/default/files/publications/OSHA4495.pdf>

US government safety guidance

Anecdotal industry material

During research, public OFM/phone-farm discussions were reviewed to understand terminology and claimed operating patterns. Self-described operators discuss old iPhone racks, multi-account posting, proxies, UI automation and other tactics. These reports are not independently verified, and some described tactics appear designed to bypass platform integrity controls. They are therefore not reproduced as implementation instructions in this guide. The architecture instead uses enterprise device-management concepts, approved APIs, human workflows and auditable networking.

Final engineering principle

Source-use matrix

Topic
Primary sources
Android enrollment/policy/ADB/VPN
S1-S4
Apple enrollment/supervision/MDM
S5-S8
Device-farm comparison
S9
Power/thermal/safety
S10-S13, S31
Proxy/DNS/CGNAT/VPN
S14-S19
Messaging/monitoring
S20-S21
TikTok publishing/policy
S22-S23
Reddit/Snap/X rules
S24-S27
OnlyFans responsibility context
S28-S30 (historical/secondary caveat)

APPENDIX M - 27 August 2026 Web Verification Register

The following first-party or maintained technical sources were used to update/correct the handbook. Platform/vendor behavior can change; re-check before implementation.

Apple - UIApplicationSupportsIndirectInputEvents

iOS 17+ defaults indirect pointing-device clicks to indirectPointer when the app does not opt out; older systems defaulted to direct-touch compatibility behavior.

<https://developer.apple.com/documentation/bundleresources/information-property-list/uiapplicationsupportsindirectinputevents>

Apple - DeviceCheck / App Attest WWDC21

DeviceCheck stores two developer-defined bits and timestamp per device on Apple servers; state survives reinstall, device transfer, and Erase All Contents and Settings. App Attest keys are per installation and do not survive reinstall.

<https://developer.apple.com/videos/play/wwdc2021/10244/>

Apple - App Attest integrity

App Attest keys remain through app updates but do not survive reinstall, migration, or restore.

<https://developer.apple.com/documentation/DeviceCheck/establishing-your-app-s-integrity>

Google - Play Integrity Device Recall

2026 beta allows app-defined per-device data to be recalled after reinstall and device reset.

<https://developer.android.com/google/play/integrity/device-recall>

Appium - XCUITest real-device preparation

Real-device XCUITest requires WebDriverAgent installed and provisioned, plus trust/developer/UI automation settings.

<https://appium.github.io/appium-xcuitest-driver/latest/preparation/real-device-config/>

Apple - iPhone Mirroring

Requires compatible Mac/iPhone, same Apple Account with 2FA, Bluetooth/Wi-Fi, phone locked and nearby; it is not a general multi-device farm control API.

<https://support.apple.com/120421>

Apple - Apple Configurator supervision

Supervision provides additional management controls and is best established before activation/on a fully erased device.

<https://support.apple.com/guide/apple-configurator-mac/supervise-devices-apd9e4f64088/mac>

Apple - Configurator Blueprints

Blueprints can package profiles/apps/actions and apply them to any number of devices.

<https://support.apple.com/guide/apple-configurator-mac/about-the-blueprint-browser-cad8be0979f/mac>

Apple - FileVault remote unlock

Apple-silicon Macs on macOS 26+ can unlock FileVault over SSH after restart if Remote Login and network are available.

<https://support.apple.com/guide/security/managing-filevault-sec8447f5049/web>

Apple - Mac mini specifications

Current M4 Mac mini starts with 16 GB unified memory and exposes Thunderbolt/USB-C connectivity; higher memory is configurable.

<https://www.apple.com/mac-mini/specs/>

Google - Chrome profiles

Profiles separate bookmarks/history/passwords/settings, but users with device access can switch profiles; signing in can synchronize data to the Google Account.

<https://support.google.com/chrome/answer/2364824>

Google - Chrome Sync data

Chrome Sync can store bookmarks, history, tabs, passwords and other profile data in a Google Account.

<https://support.google.com/chrome/a/answer/13616205>

TikTok - Business Center permissions

Business Center supports account/asset-level permissions for members and partners.

<https://ads.tiktok.com/help/article/about-assets-and-asset-level-permissions?lang=en>

X - Delegate

X Delegate supports owner/admin/contributor collaboration without password sharing; password changes do not revoke delegates.

<https://help.x.com/en/managing-your-account/how-to-use-the-delegate-feature>

RFC 6762 - Multicast DNS

mDNS is link-local multicast, so VLAN/subnet boundaries require deliberate service-discovery design.

<https://datatracker.ietf.org/doc/html/rfc6762>

APPENDIX N - Internal Source Register and Authority Notes

Phone Farm - Technical Brief (26 August 2026)

Provides the two-operator comparison, disposable versus durable models, open questions, cost modeling and the team's own n=1 Snapchat observation. Operator figures remain self-reported unless explicitly marked internal measured.

Client Account Separation - Internal Technical Reference (20 August 2026)

Provides the six-layer separation model, blast-radius framing, delegated-access principles, credential/billing/vendor controls and offboarding architecture. This handbook adopts that model as a core security principle.

IG Account Warmup - Protocol vs Mythology (25 August 2026)

Industry aggregation that intentionally records contradictory operator advice. It is used to identify hypotheses and to demonstrate why exact warmup timelines should not be encoded as facts.

iOS Reset Protocol (30 July 2026)

Provides the useful distinction between ordinary identifiers and DeviceCheck persistent state. First-party Apple documentation was used to verify/correct the mechanism. Any reset/reuse steps that could become an enforcement-evasion recipe are not operationalized in this handbook.

Farm Build - Functional Notes (26 August 2026)

Mechanism-focused research on iOS control routes, media import, host/USB limits, power and provisioning. Vendor-specific claims remain claims until bench tested.

Phone Farm Handover - Dim to Yigithan (26 August 2026)

Primary hands-on state: existing chassis/hub behavior, current phones/Windows host, Xiaowei same-network mirroring result, inbound Niazun board, Mac mini hypothesis, remote-operations requirements and open board questions. Hands-on observations are treated as authoritative for the exact tested setup.

APPENDIX O - New Analytical Insights Derived in This Edition

Pod architecture is the unifying abstraction. It bounds USB, power, thermal, network and incident risk simultaneously.

Reset semantics are a category error unless local identifiers and server-side state are separated. Both Apple and Google now provide privacy-preserving ways for developers to remember a device after reinstall/reset.

The Mac question should be split into four questions: provisioning, display, input and remote operations. "Use a Mac" is not one architecture decision.

Client separation should exist in the data model. If the backend cannot query "everything this operator/vendor can touch for Client X," offboarding and incident containment are manual guesswork.

Warmup is an experimental umbrella, not a variable. Contradictory successful routines strongly suggest hidden/confounded variables; the system should measure content/account history instead of preserving rituals.

The unsigned-vendor-software risk is comparable to giving a third party a keyboard and screen inside the account environment. Containment of the host is therefore part of the control-board BOM.

Media delivery needs reconciliation, not transfer. The success state is a verified Photos/MediaStore asset and known cleanup state, not bytes copied.

Remote desktop should be treated as the out-of-band management plane. Daily work belongs in the device control plane; this mirrors datacenter design where iLO/IPMI is not the application UI.

Capacity is a curve, not a number. "Devices per host" is meaningless without specifying screen method, frame rate, control method, task concurrency, hub topology and observability requirements.

The project should accumulate evidence as code. Bench results, ADRs and hardware-contract tests should live beside the implementation so future Claude/Codex sessions inherit measured truth rather than conversation memory.

APPENDIX P - Final Implementation Path in One Page

DAY 0

inventory everything Dim already has
create repo + evidence ledger

GATE 1: 1-2 DEVICES

test Mac capture hypothesis
test inbound board
test board + display coexistence
test media ingestion
characterize iOS input events in own test app
decide control/display architecture in ADR

GATE 2: 5 DEVICES

build one pod
node agent + fake/real adapter
inventory, health, tasks, audit, content authorization
remote recovery

GATE 3: 10 DEVICES

7-day reliability soak
power/thermal/network/USB metrics
operator workflow without physical presence

GATE 4: 20 DEVICES

only if measured capacity supports it
production-quality monitoring/security/runbooks

GATE 5: MULTI-POD

replicate known-good pod
central control plane schedules across pods
do not increase mystery density inside one host/hub

Final principle: when the research is uncertain, the output should be a test, not a confident paragraph.